Simultaneous Contact Selection and Planning for Contact-Rich Manipulation with Cascaded Optimization

***Abstract*—We propose an optimization-based framework for robust contact-rich manipulation. Recent contact-implicit methods enable online hybrid planning across contact modes, allowing closed-loop manipulation for a given target state and contact location sequence of the robot and object. However, most existing approaches lack the ability to autonomously reason and generate diverse contact location sequences and manipulation trajectories, i.e., active contact location selection, which limits their applicability to relatively simple tasks. Active contact location selection is challenging due to complementarity in contact dynamics and the sparse gradients, making the design of a unified framework for contact selection and planning difficult. To address these challenges, we introduce Simultaneous Contact Selection and Planning (SCSP), a cascaded optimization framework comprising Contact Selection Optimization (CSO) and Contact Planning Optimization (CPO). CSO leverages a surrogate contact model and discrete-continuous optimization to efficiently resolve the nonsmoothness and coupling in contact selection, enabling online search over sampled contact locations. CPO performs prior-guided contact planning by evaluating the reference contact locations produced by CSO and generating corresponding manipulation trajectories in real time for redundant manipulators. Extensive simulations and real-world experiments demonstrate that SCSP produces diverse manipulation behaviors and robust control under inaccurate dynamics and perceptual noise. We further validate the generalization of the framework on challenging manipulation tasks.**

Project website: https://sites.google.com/view/scsp-robot.

***Index Terms*—Contact-Rich Manipulation, Contact Selection, Contact-Implicit Trajectory Optimization**

NOMENCLATURE

$p, \boldsymbol{p}$	A 3D point and its position vector.
$\theta, \boldsymbol{\psi}$	Angle and quaternion vector.
$\mathcal{O}, \partial\mathcal{O}$	Object geometry and surface.
$\mathcal{R}, \partial\mathcal{R}$	Robot geometry and surface.
h	Time interval.
$\boldsymbol{\lambda}, \boldsymbol{\beta}$	Contact force and contact impulse vector.
$\boldsymbol{\phi}$	Contact distance vector.
$\boldsymbol{J}$	Contact Jacobian.
$\boldsymbol{n}, \boldsymbol{t}_i$	Normal vector and i-th tangential vector.
$\boldsymbol{x} = [\boldsymbol{p}, \boldsymbol{\psi}]$	Pose vector consists of position and orientation.
$\boldsymbol{q}, \dot{\boldsymbol{q}}, \ddot{\boldsymbol{q}}$	Robot configuration, velocity and acceleration.
$\boldsymbol{s}$	System state consists of object pose $\boldsymbol{x}_o$ and $\boldsymbol{q}$.
$\boldsymbol{v}$	Velocity vector in Cartesian space.

I. INTRODUCTION

CONTACT-Rich manipulation requires robots to intentionally make and break contact with objects to drive them toward desired states. Specifically, the robot needs to plan the contact sequence consisting of approaching trajectory, contact locations, contact forces, and transitions between different contact modes (i.e., contact, separation, sliding, and sticking). Such contact planning is challenging due to the combinatorial complexity of contact modes and the inherent nonsmoothness arising from the hybrid nature of contact dynamics. Early model-based approaches typically relied on hybrid planning, either separating continuous trajectory generation from discrete contact-mode transitions through hierarchical frameworks [1]–[5], or handling the hybrid nature of contact with mixed-integer programs [6]–[8]. However, this separation often suffers from poor scalability caused by the explosion of contact trajectories. Therefore, these methods offer limited manipulation strategies and poor robustness.

Recent advances in contact-rich manipulation have been achieved through both learning-based and model-based methods. Learning-based methods [9]–[17] directly learn mappings from object states to contact strategies, thereby enabling complex contact-rich behaviors without explicitly modeling or optimizing through the hybrid and nonsmooth contact dynamics encountered by model-based methods. However, these methods typically rely on offline training, and their generalization is often demonstrated within specific task families, such as across unseen object geometries or variations of a predefined task. For example, DyWA [17] demonstrates impressive 6D non-prehensile manipulation across diverse object geometries, i.e., cross-object generalization, but remains limited to the specific robot embodiment and task on which it is trained. Consequently, achieving broadly applicable contact-rich manipulation across substantially different robot embodiments and task settings, which we refer to as cross-task generalization in this paper, remains challenging.

Contact-implicit trajectory optimization (CITO) subsequently avoids this explicit mode decomposition by modeling robot–object interaction (ROI) with complementary contact dynamics and solving the resulting trajectory optimization problem in a unified framework. CITO was first applied to locomotion [26], [27] and later extended to manipulation. We mainly focus on Contact-Implicit Model Predictive Control (CIMPC), as its receding-horizon formulation continuously replans from the current state and is therefore particularly suitable for real-time closed-loop contact-rich manipulation. CIMPC embeds contact dynamics as constraints in a nonlinear programming (NLP) solver to compute the robot's control inputs online. CIMPC incorporates approximated time-stepping contact dynamics [28], [29] into trajectory optimization, enabling online hybrid contact planning. Because contact models commonly used in physical simulators are often formulated as linear complementarity problems (LCPs)

TABLE I
COMPARISON OF CONTACT PLANNING METHODS.

Methods	Perception	Active Contact Selection	6D Manip. Exp.	Real-Time	Irregular Geometry	Task Diversity
C3 [18]	Exact State	✗	✗	✓	✗	✓
CRISP [19]	Exact State	✗	✗	✓	✗	✗
IDTO [20]	Exact State	✗	✓	✓	✗	✓
CQDCs [21]	Exact State	✗	✓	✓	✗	✓
CF-MPC [22]	Exact State	✗	✓	✓	✓	✓
STOCS-3D [23]	Exact State	✓	✓	✗	✓	Not Validated
ContactSDF [24]	Vision	✗	✓	✓	✓	✓
DyWA [17]	Vision	✓	✓	✓	✓	✗
Sampling [25]	Vision	✓	✓	✓	✓	Not Validated
SCSP	Vision	✓	✓	✓	✓	✓

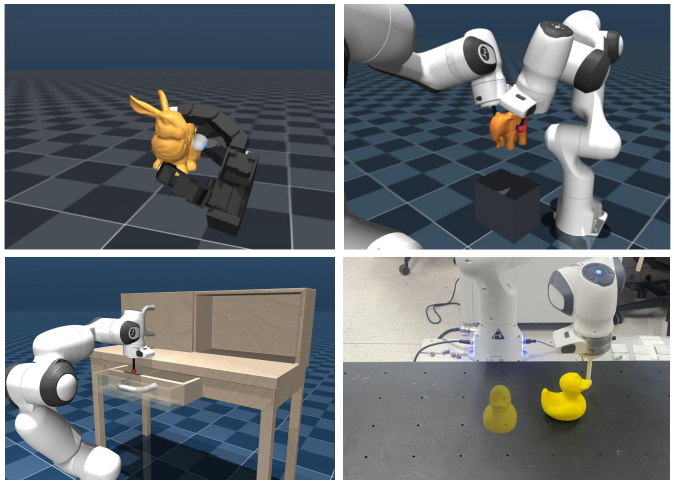


Fig. 1. SCSP enables online contact selection and closed-loop generation of diverse manipulation trajectories, allowing it to address a wide range of challenging contact-rich manipulation tasks.

or quadratic programs (QPs), directly using them can hinder efficient online optimization. CIMPC methods therefore often adopt approximate contact models that trade physical fidelity for computational efficiency. Representative methods improve the efficiency and scalability of this process through different contact-model approximations and optimization strategies [18], [21], [30]. In particular, CF-MPC [22] introduces a complementarity-free contact model to improve the efficiency of online contact-implicit MPC, substantially accelerating contact-rich optimization.

Although CIMPC enables online planning of contact-mode transitions, actively selecting contact locations on the object remains challenging. Contact geometry and complementarity constraints make the resulting optimization nonsmooth and nonconvex, while contact-free states often provide ineffective gradients toward candidate contacts. Because contact locations are typically determined implicitly by the robot trajectory, short planning horizons and local optimization can obscure the long-term benefits of reaching distant contacts. Recent methods improve contact-rich optimization and execution: Suh et al. [31] introduce a contact-trust-region formulation, Jiang et al. [32] integrate motion–contact planning with tracking, ContactSDF [24] provides differentiable multi-contact models, and CRISP [19] discovers contact sequences within contact-implicit optimization. These methods, however, do not explicitly select task-relevant robot–object contact locations; CRISP is also evaluated primarily in planar settings. Venkatesh et al. [25] address this issue by sampling contact candidates and solving a local CIMPC problem for each, but at the cost of repeated lower-level optimization. Xie et al. [33] use a hierarchical RL–MPC policy to select contacts for a low-level CIMPC controller, but require task-specific training. Efficient task-directed contact selection for closed-loop planning therefore remains unresolved.

In this work, we propose Simultaneous Contact Selection and Planning (SCSP), a cascaded optimization framework for complex contact-rich manipulation. It combines object-centric contact guidance, executable trajectory planning, and a ranking strategy that balances the guided contact with nearby feasible alternatives. Table I summarizes the scope of SCSP relative to contact-planning methods. The contributions are:

- An efficient contact selection method, CSO, is developed for online contact-guidance generation. It samples a candidate point set and performs batched optimization to avoid direct surface-gradient search and the nonsmoothness of contact selection. A lightweight surrogate contact model (SCM) enables fast evaluation of candidate utility and selection of the best contact as guidance.
- A contact planning method, CPO, is proposed with a unified cost formulation for generating approach, contact-establishment, and contact-switching trajectories.
- A ranking strategy forms the cascade between contact guidance and planning by ranking the guided contact together with nearby feasible candidates. This preserves task-directed guidance while allowing CPO to select an executable nearby contact, thereby maintaining execution flexibility and contact-trajectory diversity.
- Extensive simulation and real-world experiments validate SCSP across diverse object geometries and contact-rich tasks. Real-world experiments further demonstrate robust vision-based nonprehensile manipulation using only contact calibration and without exact object perception.

The rest of the paper is organized as follows. Section II reviews the preliminaries. The CSO and CPO are introduced in Sections III and IV, respectively. Section V presents the experimental results, and Section VI evaluates extensions of the SCSP architecture to additional contact-rich manipulation

settings. Finally, discussion and limitations are presented in Section VII, and conclusions are drawn in Section VIII.

II. Preliminaries

We begin with a brief overview of the contact-dynamics and contact-planning formulations used throughout this work. Section II-A introduces time-stepping contact dynamics and the associated complementarity formulation. Section II-B introduces model-based contact planning and reviews CIMPC as a representative online formulation. Section II-C reviews the complementarity-free contact model that motivates the lightweight contact-model approximation used later in CSO.

A. Contact Dynamics

Contact dynamics provides a physical description of interactions among the robot, object, and environment. Here, we consider the commonly used Anitescu model [29], which is formulated as the following time-stepping equation:

$$\epsilon \boldsymbol{M}_o \boldsymbol{v}_o = h\boldsymbol{\tau}_o + \sum_{i=1}^{n_c} \boldsymbol{J}_{o,i}^\top \boldsymbol{\beta}_i, \tag{1a}$$

$$h\boldsymbol{K}_r\left(h\boldsymbol{v}_r - \boldsymbol{u}\right) = h\boldsymbol{\tau}_r + \sum_{i=1}^{n_c} \boldsymbol{J}_{r,i}^\top \boldsymbol{\beta}_i. \tag{1b}$$

Here, $\boldsymbol{\beta}_i$ denotes the contact impulse over the time interval h. In the contact-force formulation below, we use $\boldsymbol{\lambda}_i$ to denote the corresponding contact force. Assuming that the contact force is approximately constant within one time step, they are related by $\boldsymbol{\beta}_i \approx h\boldsymbol{\lambda}_i$. $\epsilon\boldsymbol{M}_o$, $\boldsymbol{v}_o$, $\boldsymbol{\tau}_o$, and $\boldsymbol{J}_{o,i}$ are the regularized mass matrix, velocity change, non-contact force, and contact Jacobian of the object, respectively. $\boldsymbol{K}_r$, $\boldsymbol{v}_r$, $\boldsymbol{\tau}_r$, and $\boldsymbol{J}_{r,i}$ are the robot stiffness matrix, velocity, non-contact torque, and contact Jacobian, respectively. The control input is defined as $\boldsymbol{u} = \Delta\boldsymbol{q}$.

The above equations describe, from the perspective of momentum balance, how the robot influences the object's state through contact, as well as how the object interacts with the environment. Here, we follow [22], [34] and consider the robot to be under impedance control [35], modeling the robot–object interaction as a spring system. The contact constraints between contact forces and system motion are typically formulated as the following cone complementarity problem [36]:

$$\mathcal{K}_i \ni \boldsymbol{\lambda}_i \perp \boldsymbol{J}_i\boldsymbol{v} + \frac{1}{h}\begin{bmatrix}\phi_i \\ 0 \\ 0\end{bmatrix} \in \mathcal{K}_i^*, \qquad \forall i \in \{1,\dots,n_c\}, \tag{2}$$

where $\mathcal{K}_i = \left\{\boldsymbol{\lambda}_i \in \mathbb{R}^3 \,\middle|\, \mu_i\lambda_i^n \geq \left\|\boldsymbol{\lambda}_i^d\right\|\right\}$ is the Coulomb friction cone, and $\mathcal{K}_i^*$ is the dual cone as $\mathcal{K}_i^* = \left\{\boldsymbol{v} \,\middle|\, \boldsymbol{J}_i^n\boldsymbol{v} + \frac{\phi_i}{h} \geq \mu_i \left\|\boldsymbol{J}_i^d\boldsymbol{v}\right\|\right\}$.

The KKT conditions are [29], [37]:

$$\min_{\boldsymbol{v}} \quad \frac{1}{2}h^2\boldsymbol{v}^\top\boldsymbol{Q}\boldsymbol{v} - h\boldsymbol{v}^\top\boldsymbol{b}(\boldsymbol{u}), \tag{3a}$$

$$\text{s.t.} \quad \boldsymbol{J}_i\boldsymbol{v} + \frac{1}{h}\begin{bmatrix}\phi_i \\ 0 \\ 0\end{bmatrix} \in \mathcal{K}_i^*, \qquad i \in \{1,\dots,n_c\}. \tag{3b}$$

where $\boldsymbol{Q} \in \mathbb{R}^{(n_o+n_{\mathrm{DoF}})\times(n_o+n_{\mathrm{DoF}})}$ and $\boldsymbol{b}(\boldsymbol{u}) \in \mathbb{R}^{n_o+n_{\mathrm{DoF}}}$ are

$$\boldsymbol{Q} := \begin{bmatrix}\epsilon\boldsymbol{M}_o/h^2 & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{K}_r\end{bmatrix}, \qquad \boldsymbol{b}(\boldsymbol{u}) := \begin{bmatrix}\boldsymbol{\tau}_o \\ \boldsymbol{K}_r\boldsymbol{u} + \boldsymbol{\tau}_r\end{bmatrix}. \tag{4}$$

Here, n_o and n_{DoF} are the dimensions of the object and robot states, respectively, and n_c is the number of contacts.

Problem (3) is a second-order cone program (SOCP) and is often approximated by a QP through a polyhedral approximation of the friction cone:

$$\left(\boldsymbol{J}_i^n - \mu_i\boldsymbol{J}_{i,j}^d\right)\boldsymbol{v} + \frac{\phi_i}{h} \geq 0, \quad i \in \{1,\dots,n_c\}, \quad j \in \{1,\dots,n_d\}, \tag{5}$$

where n_d is the number of friction-cone facets. Solving (3) corresponds to one step of physically consistent contact simulation. The resulting velocity $\boldsymbol{v}$ can then be used to integrate the current state as $\boldsymbol{q}_{t+1} = \boldsymbol{q}_t \oplus h\boldsymbol{v}$. Problem (3) and its variants are widely used in physics simulators.

B. Model-based Contact Planning

Contact planning refers to generating trajectories for approaching, making contact, and breaking contact with the object to manipulate it to a target state. Model-based contact planning methods integrate contact dynamics and solve trajectory optimization problems. As a representative method, CIMPC is formulated as:

$$\begin{aligned}\min_{\boldsymbol{u}_{0:N-1}} \quad & \sum_{k=0}^{N-1} \ell(\boldsymbol{x}_k, \boldsymbol{u}_k) + V_f(\boldsymbol{x}_N) \\ \text{s.t.} \quad & \boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k, \boldsymbol{u}_k), \quad k = 0,\dots,N-1, \\ & \boldsymbol{x}_k \in \mathcal{X}, \quad k = 0,\dots,N, \\ & \boldsymbol{u}_k \in \mathcal{U}, \quad k = 0,\dots,N-1.\end{aligned} \tag{6}$$

in which $\boldsymbol{x}_k$ is the system state, including the object pose q_k and robot configuration, and f denotes the contact dynamics. The stage cost ℓ can take various forms and is typically used to guide the process of approaching the contact location. The terminal cost V_f is usually defined as the error between the object pose and the desired pose:

$$\ell_{\mathrm{pose}} = w_{\mathrm{pos}}\left\|\boldsymbol{p}_{\mathrm{obj}} - \boldsymbol{p}_{\mathrm{goal}}\right\|_2^2 + w_{\mathrm{quat}}\left(1 - \left(\boldsymbol{\psi}_{\mathrm{obj}}^\top\boldsymbol{\psi}_{\mathrm{goal}}\right)^2\right). \tag{7}$$

The hybrid nature of contact dynamics renders (6) a piecewise-smooth nonlinear complementarity problem (NCP). The full contact model is too costly for online receding-horizon optimization, so we use efficient approximations.

C. Complementarity-free Model

We next introduce the complementarity-free contact model proposed in [22], which motivates the contact-model approximation used later in CSO. Problem (3) has the dual form:

$$\begin{aligned}\max_{\boldsymbol{\beta}\geq\boldsymbol{0}} \quad & -\frac{1}{2h^2}\boldsymbol{\beta}^\top\left(\widetilde{\boldsymbol{J}}\boldsymbol{Q}^{-1}\widetilde{\boldsymbol{J}}^\top + \boldsymbol{R}\right)\boldsymbol{\beta} \\ & -\frac{1}{h}\left(\widetilde{\boldsymbol{J}}\boldsymbol{Q}^{-1}\boldsymbol{b} + \widetilde{\boldsymbol{\phi}}\right)^\top\boldsymbol{\beta} - \frac{1}{2}\boldsymbol{b}^\top\boldsymbol{Q}^{-1}\boldsymbol{b}.\end{aligned} \tag{8}$$

in which $\boldsymbol{R}$ is a small regularization term commonly used to enhance numerical stability, and

$$\widetilde{\boldsymbol{J}} = \begin{bmatrix} \boldsymbol{J}_1^n - \mu_1 \boldsymbol{J}_{1,1}^t \\ \vdots \\ \boldsymbol{J}_1^n - \mu_1 \boldsymbol{J}_{1,n_t}^t \\ \vdots \\ \boldsymbol{J}_{n_c}^n - \mu_{n_c} \boldsymbol{J}_{n_c,1}^t \\ \vdots \\ \boldsymbol{J}_{n_c}^n - \mu_{n_c} \boldsymbol{J}_{n_c,n_t}^t \end{bmatrix}, \widetilde{\boldsymbol{\phi}} = \begin{bmatrix} \phi_1 \\ \vdots \\ \phi_1 \\ \vdots \\ \phi_{n_c} \\ \vdots \\ \phi_{n_c} \end{bmatrix}, \boldsymbol{\beta} = \begin{bmatrix} \beta_{1,1} \\ \vdots \\ \beta_{1,n_d} \\ \vdots \\ \beta_{n_c,1} \\ \vdots \\ \beta_{n_c,n_d} \end{bmatrix}. \quad (9)$$

Here, $\boldsymbol{\beta}$ is not only a dual variable but also has a physical interpretation as the contact impulse. Specifically, the solution of (8) satisfies the following complementarity constraints:

$$\mathbf{0} \leq \boldsymbol{\beta} \perp \frac{1}{h}\left(\widetilde{\boldsymbol{J}}\boldsymbol{Q}^{-1}\widetilde{\boldsymbol{J}}^{\top} + \boldsymbol{R}\right)\boldsymbol{\beta} + \left(\widetilde{\boldsymbol{J}}\boldsymbol{Q}^{-1}\boldsymbol{b} + \widetilde{\boldsymbol{\phi}}\right) \geq \mathbf{0}. \quad (10)$$

Following [22], we replace the dense inverse $(\widetilde{\boldsymbol{J}}\boldsymbol{Q}^{-1}\widetilde{\boldsymbol{J}}^{\top} + \boldsymbol{R})^{-1}$ by a positive-definite diagonal matrix $\boldsymbol{K}(\boldsymbol{q})$ chosen as a local approximation. This replacement defines a modified dual problem; it is not an equality that holds for a general multi-contact geometry. With $\boldsymbol{D} = \boldsymbol{K}^{-1}$, the modified problem has the following closed-form solution:

$$\boldsymbol{\beta}^{+} = \max\left(-h\boldsymbol{K}(\boldsymbol{q})\left(\widetilde{\boldsymbol{J}}\boldsymbol{Q}^{-1}\boldsymbol{b} + \widetilde{\boldsymbol{\phi}}\right), \mathbf{0}\right). \quad (11)$$

The complementarity-free model is then written as:

$$f_{\text{cf}} : \begin{cases} \boldsymbol{v}^{+} = \frac{1}{h}\boldsymbol{Q}^{-1}\boldsymbol{b} + \frac{1}{h^2}\boldsymbol{Q}^{-1}\widetilde{\boldsymbol{J}}^{\top}\boldsymbol{\beta}^{+}, \\ \boldsymbol{x}_{k+1} = \boldsymbol{x}_k + h\left(\boldsymbol{v}_k + \boldsymbol{v}^{+}\right). \end{cases} \quad (12)$$

where $\boldsymbol{\beta}$ denotes the contact impulses of all contact points and $\boldsymbol{v}^{+}$ combines the object velocity $\boldsymbol{v}_o$ and robot velocity $\boldsymbol{v}_r$.

The modified problem is exact for the chosen diagonal surrogate, while its discrepancy from the original contact model depends on the discarded off-diagonal coupling. The SCM introduced in Section III starts from the same diagonal approximation, replaces the max operator in (11) with a smooth softplus, and partially restores the off-diagonal coupling with $K = 3$ unrolled Gauss–Seidel sweeps over the contacts; its predictive fidelity is evaluated empirically in Section V-B2.

III. Contact Selection Optimization

In this section, we present the design of CSO. We begin by analyzing the key challenges of integrating contact selection into CIMPC, which limit the diversity of existing methods. We then formulate the contact selection problem and discuss the main difficulties in solving it. Finally, we present the detailed design of our CSO module for online contact selection.

A. Why Contact Selection is necessary?

Contact selection refers to choosing object contact locations $\{\boldsymbol{p}_{\mathcal{O},i}\} \subset \partial\mathcal{O}$ that can generate task-beneficial wrenches for changing the object pose. According to (1a) and (1b), the object wrench is jointly determined by the contact Jacobian, which depends on the contact location and its local geometry, and the contact impulse $\boldsymbol{\beta}_i$ (or, equivalently, the contact force $\boldsymbol{\lambda}_i$). For some contact locations, no force within the corresponding friction cone can produce a wrench that improves the object pose; contact switching is therefore necessary.

CIMPC models robot–object interaction (ROI) with contact dynamics whose informative gradients are confined to the low-dimensional contact manifold $\mathcal{S}_{\text{contact}}$ embedded in the full state space $\mathcal{S}$. The contact-free manifold $\mathcal{S}_{\text{free}}$ provides no effective contact-induced gradient, making autonomous contact selection difficult and requiring attractive costs or reference trajectories to induce contact. Although CIMPC can choose between contact and non-contact modes, its short receding horizon may not reveal, within the fixed horizon, the benefit of reaching a distant contact location.

We formulate contact selection separately. As illustrated in Fig. 2, we ignore robot contact-location planning and solve:

$$\begin{aligned} \min_{\boldsymbol{p}_{\text{r}} \in \partial\mathcal{O}, \boldsymbol{\lambda} \in \mathcal{K}} \; & \ell_{\text{cso}}(\boldsymbol{x}_T) \\ \text{s.t.} \quad & \boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k, \boldsymbol{p}_{\text{r}}, \boldsymbol{\lambda}), \quad k = 0, \ldots, T-1. \end{aligned} \quad (13)$$

where $\mathcal{K}$ is the set of feasible contact forces and $\ell_{\text{cso}}(\cdot)$ is the objective of contact selection. $\boldsymbol{p}_{\text{r}} \in \mathbb{R}^{3\times n_{\text{r}}}$ represents the locations where the robot applies contacts to the object, with n_{r} denotes the number of robot contacts. (13) addresses the gradient sparsity issue arising from hybrid planning in the original contact-rich manipulation problem by defining CSO to solve for low-cost sampled contact locations and contact forces on $\mathcal{S}_{\text{contact}}$. The neglected process of the robot approaching $\mathcal{S}_{\text{contact}}$ from its current state is the primary source of the gradient sparsity. Note that (13) is hard to solve due to the coupling of contact location $\boldsymbol{p}_{\text{r}}$ and the contact force $\boldsymbol{\lambda} = [\boldsymbol{\lambda}_{\text{env}}, \boldsymbol{\lambda}_{\text{r}}]$. Therefore, we adopt a commonly used simplification for handling nonconvex optimization problems to reformulate (13) as subproblems:

$$\min_{\boldsymbol{p}_{\text{r}} \in \partial\mathcal{O}} \quad V(\boldsymbol{p}_{\text{r}}), \quad (14)$$

in which:

$$\begin{aligned} V(\boldsymbol{p}_{\text{r}}) := \min_{\boldsymbol{\lambda}_{\text{r}} \in \mathcal{K}} \; & \ell_{\text{cso}}(\boldsymbol{x}_T) \\ \text{s.t.} \quad & \boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k, \boldsymbol{p}_{\text{r}}, \boldsymbol{\lambda}_{\text{r}}, \boldsymbol{\lambda}_{\text{env}}), \quad k = 0, \ldots, T-1. \end{aligned} \quad (15)$$

The physical interpretation of (14) and (15) is that a contact location is regarded as optimal if applying the best possible combination of contact force $\boldsymbol{\lambda}^{*}$ yields the minimum motion cost of the object. Despite the simplifications, (14) remains a complex problem with nonsmooth constraints $\boldsymbol{p}_{\text{r}} \in \partial\mathcal{O}$ that depend on the object geometry. Meanwhile, (15) is a complementary problem with coupled environment forces $\boldsymbol{\lambda}_{\text{env}}$ and robot contact forces $\boldsymbol{\lambda}_{\text{r}}$. The nonconvexity makes online CSO intractable. We next present the solutions to further simplify both (14) and (15) to improve computational efficiency.

B. Sampling and Exchange Method

The nonsmoothness on the object geometry leads to discontinuities in the gradient of (14). The inherently nonsmooth nature of the problem motivates us to adopt a sampling-based method. To simplify (14), we uniformly sample a set

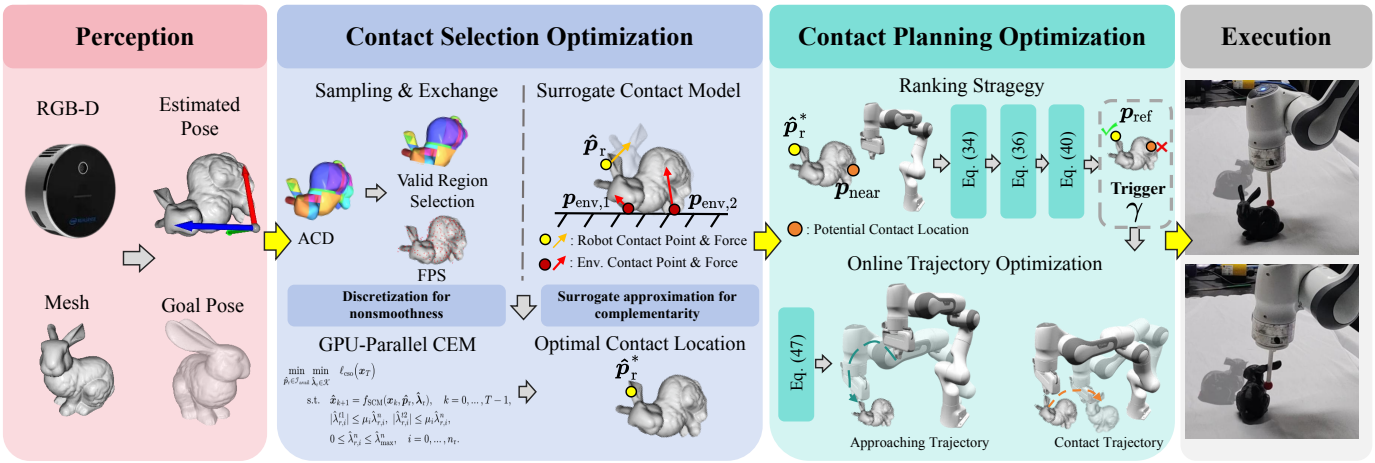


Fig. 2. Framework structure of SCSP. SCSP consists of Contact Selection Optimization (CSO) and Contact Planning Optimization (CPO). CSO discretizes nonsmooth geometry through the Sampling & Exchange method and introduces a surrogate contact model for efficient approximation of contact dynamics. CSO online computes the optimal contact location among candidate contact set $\hat{\boldsymbol{p}}_{\mathrm{r}}^{*}$ as contact prior. The ranking strategy in CPO further evaluates whether the nearest point $\boldsymbol{p}_{\mathrm{near}}$ can serve as a sufficiently competitive suboptimal substitute for $\hat{\boldsymbol{p}}_{\mathrm{r}}^{*}$ as the reference contact location, after which trajectory optimization is solved online to generate the approach and contact trajectories.

of candidate contact point set $\mathcal{I}_{\mathrm{obj}} = \{\boldsymbol{p}_i\}_{i=1}^{n_s}$ on the object surface $\partial\mathcal{O}$, thus reformulate (14) as:

$$\min_{\boldsymbol{p}_{\mathrm{r}} \in \mathcal{I}_{\mathrm{obj}}} \quad V(\boldsymbol{p}_{\mathrm{r}}), \tag{16}$$

When the sampling is sufficiently uniform, (16) provides a good approximation to (14) and is not affected by discontinuous gradients caused by the geometry. To obtain a high quality candidate set, $\mathcal{I}_{\mathrm{obj}}$ should satisfy the following conditions:

- **Physical reachability**, which requires that candidate contact locations be reachable by the robot, i.e., within the robot workspace and not occluded.
- **Contact stability**, which requires avoiding regions with high curvature where stable contact is difficult to apply.
- **Uniform distribution**, which means the sampled points are representative of approximating the original geometry.

These conditions require the sampling method to maintain uniform coverage while prioritizing manipulability over local geometric details. To address this, we propose an efficient sampling strategy as shown in Algorithm 1. We first perform approximate convex decomposition (ACD) [38] to obtain a convex approximation $\mathcal{F}$ of the original geometry $\mathcal{O}$. ACD decomposes the object into convex polygons, mitigating interference caused by local geometric details and reducing computational cost. Moreover, these polygons are typically larger and flatter, which implies greater contact stability and improved reachability.

Then we perform Farthest-Point Sampling (FPS) on the convex approximation to uniformly sample points on the polygons. Starting from an arbitrary point as a sample, the algorithm iteratively selects the points that maximize the minimal Euclidean distance to all previously chosen samples:

$$p'_{k+1} = \arg\max_{p_i \in \mathcal{F}} \min_{p'_j \in \mathcal{I}_k} \|\boldsymbol{p}_i - \boldsymbol{p}'_j\|^2, \tag{17}$$

where $\mathcal{I}_k$ is the set of already selected samples. The sampling process is performed offline. Solving (17) yields a uniformly distributed set of points $\mathcal{I}_{\mathrm{obj}}$. For each sampled point $p'_j \in \mathcal{I}_k$, we also construct its local coordinate frame to obtain the tangential and normal vectors as local contact information. The local orthonormal frame is constructed as $(\boldsymbol{n}_i, \boldsymbol{t}_{1,i}, \boldsymbol{t}_{2,i})$ in which the normal vector $\boldsymbol{n}_i$ is obtained from the mesh or point-cloud normal estimate, and two orthogonal tangent directions are computed as:

$$\boldsymbol{t}_{1,i} = \frac{\boldsymbol{a} - (\boldsymbol{a}\cdot\boldsymbol{n}_i)\boldsymbol{n}_i}{\|\boldsymbol{a} - (\boldsymbol{a}\cdot\boldsymbol{n}_i)\boldsymbol{n}_i\|}, \ \boldsymbol{t}_{2,i} = \boldsymbol{n}_i \times \boldsymbol{t}_{1,i}, \tag{18}$$

where $\boldsymbol{a}$ is an arbitrary fixed axis. When the reference axis a is nearly parallel to the surface normal n_i, we select another coordinate axis as the reference axis to avoid numerical singularity. We use this local geometric information to compute the Jacobian required for contact force optimization. Furthermore, a KD-tree $\mathcal{T} = \{(\boldsymbol{p}_i, \boldsymbol{n}_i, \boldsymbol{t}_{1,i}, \boldsymbol{t}_{2,i})\}_{i=1}^{n_s}$ is constructed over the set $\mathcal{I}_{\mathrm{obj}}$ to enable efficient nearest-neighbor queries.

An online Valid Region Selection procedure is further applied to identify feasible and task-relevant surface regions, filtering out unreachable or low-quality points such as occluded regions, regions outside the robot workspace, and regions that violate task-specific semantic masks (for example, handles or forbidden surfaces). The valid region is maintained by applying a mask to $\mathcal{F}$ to obtain $\mathcal{F}_{\mathrm{v}}$, and then selecting points on $\mathcal{F}_{\mathrm{v}}$ to obtain $\mathcal{I}_{\mathrm{avail}}$. To better illustrate the process, consider the toy example shown in Fig. 3. First, following the procedure illustrated in Fig. 3(a), we iteratively apply (17) to obtain a set of uniformly distributed sampling points over the processed object mesh. We then partition the surface online into a set of valid regions and masked regions, and select the points that lie within the valid regions to form the candidate set $\mathcal{I}_{\mathrm{avail}}$, as shown in Fig. 3(b). The Exchange step applies these masks online and exchanges $\mathcal{F}$ for the retained valid surface $\mathcal{F}_{\mathrm{v}}$; its cost is reported in Table IX. Both masks are specified offline with the object model; no online segmentation is performed.

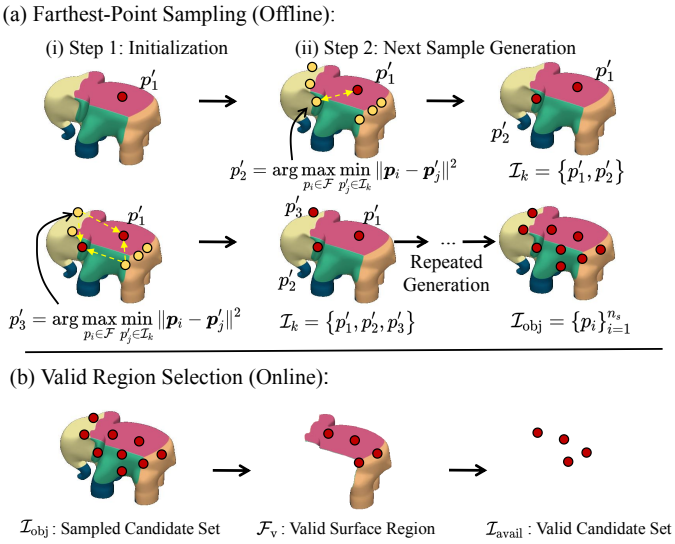


Fig. 3. The sampling strategy in CSO. The candidate point set is sampled sequentially through: (a) Farthest-Point Sampling to uniformly sample points on the object, and (b) Valid Region Selection to select valid points for contact.

Algorithm 1 Sampling and Exchange Method

Input: Object geometry $\mathcal{O}$, sample count K, fixed axis $\boldsymbol{a}$
Output: $\mathcal{T} = \{(\boldsymbol{p}_i, \boldsymbol{n}_i, \boldsymbol{t}_{1,i}, \boldsymbol{t}_{2,i})\}_{i=1}^{n_s}$ and $\mathcal{I}_{\text{avail}}$.

$\mathcal{F} \leftarrow \text{ACD}(\mathcal{O})$ ▷ Convex approximation
Initialize $\mathcal{I}_k \leftarrow \{p_1'\}$ with an arbitrary seed in $\mathcal{F}$
for $k = 1$ to $K - 1$ **do**
  $p_{k+1}' \leftarrow \arg\max_{p_i' \in \mathcal{F}} \min_{p_j \in \mathcal{I}_k} \|\boldsymbol{p}_i' - \boldsymbol{p}_j'\|^2$
  $\mathcal{I}_{k+1} \leftarrow \mathcal{I}_k \cup \{p_{k+1}'\}$ ▷ Obtain candidate set $\mathcal{I}_{\text{obj}} = \mathcal{I}_K$
end for
$\mathcal{F}_{\text{v}} \leftarrow \text{Exchange}(\mathcal{F})$ ▷ Apply offline-specified rule and manual masks
for all $p_i \in \mathcal{I}_{\text{obj}}$ **do**
  if $p_i \notin \mathcal{F}_{\text{v}}$ **then**
    Remove p_i from $\mathcal{I}_{\text{obj}}$ ▷ Valid region selection
    continue
  end if ▷ Obtain $\mathcal{I}_{\text{avail}}$
  Estimate normal $\boldsymbol{n}_i$ from $\mathcal{F}$ at p_i
  $\boldsymbol{t}_{1,i} \leftarrow \dfrac{\boldsymbol{a} - (\boldsymbol{a} \cdot \boldsymbol{n}_i)\boldsymbol{n}_i}{\|\boldsymbol{a} - (\boldsymbol{a} \cdot \boldsymbol{n}_i)\boldsymbol{n}_i\|}$, $\boldsymbol{t}_{2,i} \leftarrow \boldsymbol{n}_i \times \boldsymbol{t}_{1,i}$
end for
return $\mathcal{T} = \{(\boldsymbol{p}_i, \boldsymbol{n}_i, \boldsymbol{t}_{1,i}, \boldsymbol{t}_{2,i})\}_{i=1}^{n_s}$ and $\mathcal{I}_{\text{avail}}$

C. Surrogate Contact Model

The sampling strategy transforms the original optimization into a discrete–continuous problem. We further simplify (15) to improve efficiency. Most existing contact models are computationally expensive for scenarios that require repeatedly solving (15). Moreover, complementarity and coupling between $\boldsymbol{\lambda}_{\text{env}}$ and $\boldsymbol{\lambda}_{\text{r}}$ make the optimization difficult to converge, leading to physically inconsistent results. To enable online contact selection, we introduce a surrogate contact model (SCM); its procedure is illustrated in Fig. 4. The approximation uses the following assumptions:

Assumption 1 (Fixed Contact Set)**.** The object–environment contact location remains unchanged in each SCM rollout, i.e., no new contacts are created and existing contacts remain, so that the contact gap satisfies $\tilde{\boldsymbol{\phi}}_{\text{env}} \equiv \mathbf{0}$ throughout the rollout.

Assumption 2 (Multi-Contact Decoupling)**.** The contact between the object and the environment can be approximated by independent spring–damper systems, neglecting higher-order dynamic couplings between different contact locations. This assumption is used only to replace each Delassus block by its diagonal; the $K = 3$ Gauss–Seidel sweeps in (27) partially restore the coupling between contacts.

We reformulate the object dynamics in (1a) at the velocity level, retaining the current object velocity $\boldsymbol{v}_k$ so that the model matches the dynamic rollout used in CSO, where $\boldsymbol{M}_o$ is the object mass matrix and $\boldsymbol{v}_o^+$ is the post-step object velocity:

$$\boldsymbol{M}_o \boldsymbol{v}_o^+ = \boldsymbol{M}_o \boldsymbol{v}_k + h(\boldsymbol{\tau}_o + \sum_{i=1}^{n_{\text{r}}} \boldsymbol{J}_{\text{r},i}^\top \boldsymbol{\lambda}_{\text{r},i} + \sum_{j=1}^{n_{\text{env}}} \tilde{\boldsymbol{J}}_{\text{env},j}^\top \boldsymbol{\lambda}_{\text{env},j}), \tag{19}$$

and we denote the force-level free term $\tilde{\boldsymbol{b}}_o = \boldsymbol{M}_o \boldsymbol{v}_k / h + \boldsymbol{\tau}_o + \sum_{i=1}^{n_{\text{r}}} \boldsymbol{J}_{\text{r},i}^\top \boldsymbol{\lambda}_{\text{r},i}$, which includes the momentum term $\boldsymbol{M}_o \boldsymbol{v}_k / h$, so that (19) reads $\boldsymbol{M}_o \boldsymbol{v}_o^+ = h(\tilde{\boldsymbol{b}}_o + \sum_{j=1}^{n_{\text{env}}} \tilde{\boldsymbol{J}}_{\text{env},j}^\top \boldsymbol{\lambda}_{\text{env},j})$. The environment contact force at contact j is expressed in its contact frame as $\boldsymbol{\lambda}_{\text{env},j} = (\lambda_{\text{env},j}^n, \lambda_{\text{env},j}^{t1}, \lambda_{\text{env},j}^{t2}) \in \mathcal{K}_j$, where $\mathcal{K}_j = \{\lambda^n \geq 0,\ |\lambda^{t1}| \leq \mu_j \lambda^n,\ |\lambda^{t2}| \leq \mu_j \lambda^n\}$ is the Coulomb box used for the robot contacts in (29), with the contact Jacobian $\tilde{\boldsymbol{J}}_{\text{env},j} \in \mathbb{R}^{3\times 6}$ whose rows are $\boldsymbol{J}_j^n$, $\boldsymbol{J}_j^{t1}$, $\boldsymbol{J}_j^{t2}$; we write $\mathcal{K} = \prod_j \mathcal{K}_j$. Similar to (8), the physically consistent environment contact force can be obtained by solving the following dual problem (written in force units by dividing the impulse-level dual by h^2):

$$\begin{aligned} \max_{\boldsymbol{\lambda}_{\text{env}} \in \mathcal{K}} \quad & -\frac{1}{2} \boldsymbol{\lambda}_{\text{env}}^\top \left(\tilde{\boldsymbol{J}}_{\text{env}} \boldsymbol{M}_o^{-1} \tilde{\boldsymbol{J}}_{\text{env}}^\top + \boldsymbol{R} \right) \boldsymbol{\lambda}_{\text{env}} \\ & - (\tilde{\boldsymbol{J}}_{\text{env}} \boldsymbol{M}_o^{-1} \tilde{\boldsymbol{b}}_o + \tilde{\boldsymbol{\phi}}/h^2)^\top \boldsymbol{\lambda}_{\text{env}} - \frac{1}{2} \tilde{\boldsymbol{b}}_o^\top \boldsymbol{M}_o^{-1} \tilde{\boldsymbol{b}}_o, \end{aligned} \tag{20}$$

and the solution satisfies the cone-complementarity condition

$$\mathcal{K} \ni \boldsymbol{\lambda}_{\text{env}} \perp \boldsymbol{W}_{\text{env}} \boldsymbol{\lambda}_{\text{env}} + \boldsymbol{g}_{\text{env}} \in \mathcal{K}^*. \tag{21}$$

where

$$\boldsymbol{W}_{\text{env}} = \left(\tilde{\boldsymbol{J}}_{\text{env}} \boldsymbol{M}_o^{-1} \tilde{\boldsymbol{J}}_{\text{env}}^\top + \boldsymbol{R} \right), \quad \boldsymbol{g}_{\text{env}} = \left(\tilde{\boldsymbol{J}}_{\text{env}} \boldsymbol{M}_o^{-1} \tilde{\boldsymbol{b}}_o + \tilde{\boldsymbol{\phi}}/h^2 \right). \tag{22}$$

Here $\boldsymbol{W}_{\text{env}}$ is the Delassus matrix associated with object–environment contact. Since the contact force at each contact point is represented by its three contact-frame components $\boldsymbol{\lambda}_{\text{env},i} \in \mathbb{R}^3$, the Delassus matrix $\boldsymbol{W}_{\text{env}}$ admits a block structure

$$\boldsymbol{W}_{\text{env}} = \begin{bmatrix} \boldsymbol{W}_{11} & \boldsymbol{W}_{12} & \cdots & \boldsymbol{W}_{1n_{\text{env}}} \\ \boldsymbol{W}_{21} & \boldsymbol{W}_{22} & \cdots & \boldsymbol{W}_{2n_{\text{env}}} \\ \vdots & \vdots & \ddots & \vdots \\ \boldsymbol{W}_{n_{\text{env}}1} & \boldsymbol{W}_{n_{\text{env}}2} & \cdots & \boldsymbol{W}_{n_{\text{env}}n_{\text{env}}} \end{bmatrix},$$

where each block $\boldsymbol{W}_{ij} \in \mathbb{R}^{3\times 3}$ is

$$\boldsymbol{W}_{ij} = \tilde{\boldsymbol{J}}_{\text{env},i} \boldsymbol{M}_o^{-1} \tilde{\boldsymbol{J}}_{\text{env},j}^\top + \boldsymbol{R}_{ij}.$$

The diagonal blocks $\boldsymbol{W}_{ii}$ describe local effective inertia, while the off-diagonal blocks $\boldsymbol{W}_{ij}$ $(i \neq j)$ represent coupling

between contacts. Under Assumption 2, we neglect these couplings and decompose (21) into contact-wise subproblems:

$$\mathcal{K}_i \ni \boldsymbol{\lambda}_{\text{env},i} \perp \boldsymbol{W}_{ii}\boldsymbol{\lambda}_{\text{env},i} + (\tilde{\boldsymbol{J}}_{\text{env},i}\boldsymbol{M}_o^{-1}\tilde{\boldsymbol{b}}_o + \tilde{\boldsymbol{\phi}}_i/h^2) \in \mathcal{K}_i^*. \quad (23)$$

We approximate each diagonal block $\boldsymbol{W}_{ii}$ by a diagonal matrix $\boldsymbol{D}$, with a small residual $\boldsymbol{E}$. Under this approximation, (23) has the smooth approximate closed-form solution below. Here, $\text{softplus}_T(x) = T\log(1+e^{x/T})$ is a componentwise smooth (C^∞) approximation of $\max(x,0)$ with smoothing parameter $T>0$; its derivative is the logistic function $\sigma(x/T) = 1/(1+e^{-x/T})$. The temperature T is specified in impulse units (N s), so the force-level relation uses T/h. The smooth projection $\Pi_{\mathcal{K}_i}^T$ onto $\mathcal{K}_i$ applies softplus_T to the normal component and $\text{clip}_T(x,c) = x - \text{softplus}_T(x-c) + \text{softplus}_T(-x-c)$, a C^∞ approximation of the projection onto $[-c,c]$, to each tangential component with $c = \mu_i \hat{\lambda}^n_{\text{env},i}$:

$$\hat{\boldsymbol{\lambda}}_{\text{env},i} = \Pi_{\mathcal{K}_i}^{T/h}\left(-\boldsymbol{D}^{-1}(\tilde{\boldsymbol{J}}_{\text{env},i}\boldsymbol{M}_o^{-1}\tilde{\boldsymbol{b}}_o + \tilde{\boldsymbol{\phi}}_i/h^2)\right). \quad (24)$$

With (24), we can obtain the following lemma:

Lemma 1. Under Assumption 2, the environment contact force $\boldsymbol{\lambda}_{\text{env}}$ is an explicit function of $\boldsymbol{\lambda}_{\text{r}}$.

To obtain a high-quality approximation, we set the diagonal matrix to:

$$\boldsymbol{D} = \text{diag}(\tilde{\boldsymbol{J}}_{\text{env},i}\boldsymbol{M}_o^{-1}\tilde{\boldsymbol{J}}_{\text{env},i}^\top) + \epsilon\boldsymbol{I}, \quad (25)$$

where ϵ is the regularization coefficient, so that $\boldsymbol{D} = \text{diag}(\boldsymbol{W}_{ii})$ and the same regularization appears in $\boldsymbol{W}_{\text{env}}$. (25) gives a diagonal approximation to $\boldsymbol{W}_{ii}$ in the Frobenius norm. When the contact set changes within the CSO horizon, variations in $\tilde{\boldsymbol{J}}_{\text{env},i}$ and $\tilde{\boldsymbol{\phi}}_i$ introduce non-smoothness. Assumption 1 then gives:

Lemma 2. The initial diagonal estimate of the environment contact force is a smooth, closed-form function of $\boldsymbol{\lambda}_{\text{r}}$:

$$\hat{\boldsymbol{\lambda}}_{\text{env},i} = \Pi_{\mathcal{K}_i}^{T/h}(-\boldsymbol{D}^{-1}\tilde{\boldsymbol{J}}_{\text{env},i}\boldsymbol{M}_o^{-1}\tilde{\boldsymbol{b}}_o), \quad (26)$$

with $\tilde{\boldsymbol{b}}_o$ evaluated at the robot contact forces $\hat{\boldsymbol{\lambda}}_{\text{r}}$.

Proof. Assumption 1 provides the condition $\tilde{\boldsymbol{\phi}}_i \equiv \boldsymbol{0}$ for every environment contact i during the rollout. Substituting $\tilde{\boldsymbol{\phi}}_i = \boldsymbol{0}$ and the definition of $\tilde{\boldsymbol{b}}_o$ into (24) removes the gap term and expresses the argument of $\Pi_{\mathcal{K}_i}^{T/h}$ as an affine function of the robot contact forces, which gives (26). Moreover, since the contact geometry set remains invariant over the time interval, $\tilde{\boldsymbol{J}}_{\text{env},i}$ is a constant matrix, thus $\boldsymbol{W}_{ii}$ is constant, and $\boldsymbol{D}$, $\boldsymbol{M}_o^{-1}$ are all constant matrices. Therefore, the fixed-contact map is the composition of an affine map with the C^∞ function $\Pi_{\mathcal{K}_i}^{T/h}$ and is smooth in $\boldsymbol{\lambda}_{\text{r}}$; nonsmoothness arises only when the contact set changes.

To partially compensate for the neglected inter-contact coupling, we refine the diagonal estimate with $K = 3$ unrolled Gauss–Seidel sweeps over the contacts in a fixed order: the force of each contact is computed from the object velocity already updated by the preceding contacts and by the previous sweeps. We retain only the nonnegative part of the signed contact distance, $\boldsymbol{d}_{\text{env}} = \max(\tilde{\boldsymbol{\phi}}_{\text{env}}, \boldsymbol{0})$, to avoid strong repulsion from instantaneous penetration, trading some physical fidelity for greater robustness to contact estimation errors and improved optimization behavior; for the touching contacts of Assumption 1, $\boldsymbol{d}_{\text{env}} = \boldsymbol{0}$, and $\boldsymbol{d}_{\text{env}}$ is evaluated once from the current state $\boldsymbol{x}_k$, independently of $\boldsymbol{\lambda}_{\text{r}}$. Let $\boldsymbol{v}_{\text{free}} = h\boldsymbol{M}_o^{-1}\tilde{\boldsymbol{b}}_o = \boldsymbol{v}_k + h\boldsymbol{M}_o^{-1}(\boldsymbol{\tau}_o + \sum_{i=1}^{n_{\text{r}}}\boldsymbol{J}_{\text{r},i}^\top\boldsymbol{\lambda}_{\text{r},i})$ be the free-motion velocity, i.e., the post-step object velocity without environment contact. Starting from $\hat{\boldsymbol{\lambda}}_{\text{env}}^{(0)} = \boldsymbol{0}$ and $\boldsymbol{v}^{(1,0)} = \boldsymbol{v}_{\text{free}}$ and using $\boldsymbol{d}_{\text{env}}$ in place of $\tilde{\boldsymbol{\phi}}_{\text{env}}$ in (24), sweep $k = 1,\dots,K$ computes for $i = 1,\dots,n_{\text{env}}$

$$\begin{cases} \hat{\boldsymbol{\lambda}}_{\text{env},i}^{(k)} = \Pi_{\mathcal{K}_i}^{T/h}\left(\hat{\boldsymbol{\lambda}}_{\text{env},i}^{(k-1)} - \boldsymbol{D}^{-1}\boldsymbol{r}_i^{(k)}\right), \\ \boldsymbol{r}_i^{(k)} = \tilde{\boldsymbol{J}}_{\text{env},i}\boldsymbol{v}^{(k,i-1)}/h + \boldsymbol{d}_{\text{env},i}/h^2 + \epsilon\hat{\boldsymbol{\lambda}}_{\text{env},i}^{(k-1)}, \\ \boldsymbol{v}^{(k,i)} = \boldsymbol{v}^{(k,i-1)} + h\boldsymbol{M}_o^{-1}\tilde{\boldsymbol{J}}_{\text{env},i}^\top(\hat{\boldsymbol{\lambda}}_{\text{env},i}^{(k)} - \hat{\boldsymbol{\lambda}}_{\text{env},i}^{(k-1)}), \end{cases} \quad (27)$$

where $\boldsymbol{v}^{(k+1,0)} = \boldsymbol{v}^{(k,n_{\text{env}})}$ and $\boldsymbol{D}$ is the diagonal of the Delassus block $\boldsymbol{W}_{ii}$ in (25), so that each contact performs one projected step of (24) on the velocity $\boldsymbol{v}^{(k,i-1)}$ left by the preceding contacts and sweeps ($\boldsymbol{v}^{(1,0)}/h = \boldsymbol{M}_o^{-1}\tilde{\boldsymbol{b}}_o$). The stacked force $\hat{\boldsymbol{\lambda}}_{\text{env}} = \hat{\boldsymbol{\lambda}}_{\text{env}}^{(K)}$ is the environment contact force output by SCM and is used to predict object motion in (28). With $\boldsymbol{R} = \epsilon\boldsymbol{I}$, this computation avoids forming or inverting the full Delassus matrix. Because $\boldsymbol{D}$ and $\boldsymbol{d}_{\text{env}}$ are constant for a fixed contact set and $\Pi_{\mathcal{K}_i}^{T/h}$ is C^∞, the unrolled sweeps are an explicit finite composition of affine maps and smooth projections, so the map $\boldsymbol{\lambda}_{\text{r}} \mapsto \hat{\boldsymbol{\lambda}}_{\text{env}}$ is smooth, and the only remaining nonsmoothness is the change of the contact set between rollout steps or CSO updates.

Substituting the Gauss–Seidel force from (27) into (19), we obtain a lightweight model:

$$f_{\text{SCM}}: \begin{cases} \hat{\boldsymbol{v}}^+ = h\boldsymbol{M}_o^{-1}\tilde{\boldsymbol{b}}_o + h\boldsymbol{M}_o^{-1}\sum_{i=1}^{n_{\text{env}}}\tilde{\boldsymbol{J}}_{\text{env},i}^\top\hat{\boldsymbol{\lambda}}_{\text{env},i}, \\ \hat{\boldsymbol{x}}_{k+1} = \boldsymbol{x}_k \oplus h\hat{\boldsymbol{v}}^+, \end{cases} \quad (28)$$

where $\hat{\boldsymbol{v}}^+$ is the predicted post-step object velocity (it already contains $\boldsymbol{v}_k$ through $\tilde{\boldsymbol{b}}_o$), and $\oplus$ denotes the pose update with the quaternion increment.

This Surrogate Contact Model converts the original contact dynamics into a computationally efficient formulation, and enables the continuous force subproblem to be solved by GPU-parallel CEM for each sampled candidate.

D. Formulation of CSO

Integrating SCM into (15), we can obtain the formulation of CSO as:

$$\begin{aligned} \min_{\hat{\boldsymbol{p}}_{\text{r}} \in \mathcal{I}_{\text{avail}}} \min_{\hat{\boldsymbol{\lambda}}_{\text{r}} \in \mathcal{K}} \quad & \ell_{\text{cso}}(\boldsymbol{x}_T) \\ \text{s.t.} \quad & \hat{\boldsymbol{x}}_{k+1} = f_{\text{SCM}}(\boldsymbol{x}_k, \hat{\boldsymbol{p}}_{\text{r}}, \hat{\boldsymbol{\lambda}}_{\text{r}}), \quad k = 0,\dots,T-1, \\ & |\hat{\lambda}^{t1}_{r,i}| \le \mu_i\hat{\lambda}^n_{r,i}, \ |\hat{\lambda}^{t2}_{r,i}| \le \mu_i\hat{\lambda}^n_{r,i}, \\ & 0 \le \hat{\lambda}^n_{r,i} \le \hat{\lambda}^n_{\max}, \quad i = 1,\dots,n_{\text{r}}. \end{aligned} \quad (29)$$

By solving this discrete-continuous optimization problem, we can obtain the optimal contact location among candidate contact set $\hat{\boldsymbol{p}}_r^* \in \mathcal{I}_{\text{avail}}$ as guidance for the lower-level contact planning. The CSO objective ℓ_{cso} is the object pose cost ℓ_{pose} in (7); the task-specific weights are given in Section V-A.

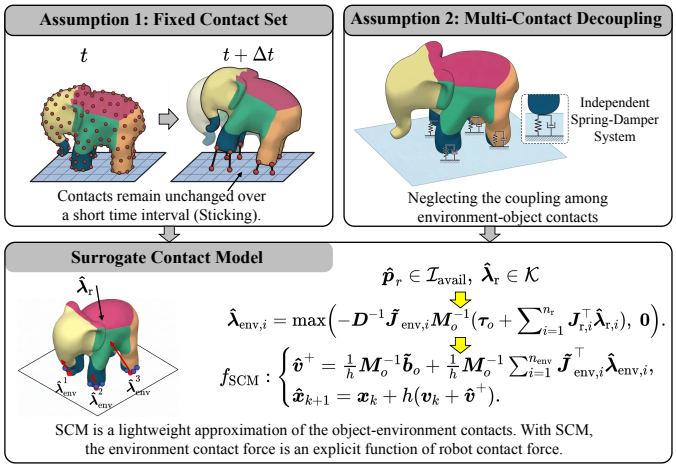


Fig. 4. Illustration of SCM. SCM is a lightweight approximation of the object-environment contact, enabling fast contact selection optimization.

The contact dynamics in (15) are non-differentiable because of complementarity constraints and changes in the contact set. With the SCM, (29) becomes a finite candidate search with a continuous force-optimization subproblem for each candidate. For a fixed contact set, differentiating (28) with respect to the robot contact force gives

$$\frac{\partial \hat{\boldsymbol{x}}_{k+1}}{\partial \boldsymbol{\lambda}_{\text{r},i}} = h^2 \boldsymbol{M}_o^{-1}\left(\boldsymbol{J}_{\text{r},i}^{\top} + \tilde{\boldsymbol{J}}_{\text{env}}^{\top}\frac{\partial \hat{\boldsymbol{\lambda}}_{\text{env}}}{\partial \boldsymbol{\lambda}_{\text{r},i}}\right), \tag{30}$$

where the first term is the direct robot-force effect and the second is the induced environment-force effect. The latter follows by differentiating the sweeps in (27): with $\boldsymbol{\Lambda}_j^{(k)} = \partial \hat{\boldsymbol{\lambda}}_{\text{env},j}^{(k)}/\partial \boldsymbol{\lambda}_{\text{r},i}$, $\boldsymbol{V}^{(k,j)} = \partial \boldsymbol{v}^{(k,j)}/\partial \boldsymbol{\lambda}_{\text{r},i}$, and $\boldsymbol{\Sigma}_j^{(k)}$ the Jacobian of $\Pi_{\mathcal{K}_j}^{T/h}$ at its argument in sweep k,

$$\boldsymbol{\Lambda}_j^{(k)} = \boldsymbol{\Sigma}_j^{(k)}\left[(\boldsymbol{I} - \epsilon \boldsymbol{D}^{-1})\boldsymbol{\Lambda}_j^{(k-1)} - h^{-1}\boldsymbol{D}^{-1}\tilde{\boldsymbol{J}}_{\text{env},j}\boldsymbol{V}^{(k,j-1)}\right], \tag{31}$$

$\boldsymbol{V}^{(k,j)} = \boldsymbol{V}^{(k,j-1)} + h\boldsymbol{M}_o^{-1}\tilde{\boldsymbol{J}}_{\text{env},j}^{\top}(\boldsymbol{\Lambda}_j^{(k)} - \boldsymbol{\Lambda}_j^{(k-1)})$, in the sweep order of (27) from $\boldsymbol{\Lambda}_j^{(0)} = \boldsymbol{0}$ and $\boldsymbol{V}^{(1,0)} = h\boldsymbol{M}_o^{-1}\boldsymbol{J}_{\text{r},i}^{\top}$; stacking $\boldsymbol{\Lambda}_j^{(K)}$ gives $\partial \hat{\boldsymbol{\lambda}}_{\text{env}}/\partial \boldsymbol{\lambda}_{\text{r},i}$. The SCM rollout is thus smooth in the robot contact forces for a fixed contact set; GPU-parallel CEM uses forward evaluations of (28), and the gradients support alternative gradient solvers.

IV. Contact Planning Optimization

In this section, we introduce the CPO module for contact planning. CPO accepts the optimal contact location predicted by CSO as reference, and generates approaching and contact trajectories online. We will discuss the problem of designing a prior-guided contact planning. Then we design a ranking strategy as an expressive connection between contact planning and contact prior from contact selection module. Finally, we propose a novel objective formulation for prior-guided contact trajectory generation, and present two solvers for CPO.

A. Problem Statement

Compared to CIMPC methods, CPO must additionally consider the guidance from the high-level module and generate the corresponding executable trajectory. Based on (6), we define CPO as the single shooting problem:

$$\begin{aligned} \min_{\boldsymbol{u}_{0:N-1}} \quad & \textstyle\sum_{k=0}^{N-1}\ell_{\text{cpo}}(\boldsymbol{x}_k, \boldsymbol{u}_k, \hat{\boldsymbol{p}}_{\text{r}}^{*}) + V_f(\boldsymbol{x}_N) \\ \text{s.t.} \quad & \boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k, \boldsymbol{u}_k), \quad k = 0, \ldots, N-1, \\ & \boldsymbol{x}_k \in \mathcal{X}, \quad k = 0, \ldots, N, \\ & \boldsymbol{u}_k \in \mathcal{U}, \quad k = 0, \ldots, N-1. \end{aligned} \tag{32}$$

Here $\hat{\boldsymbol{p}}_{\text{r}}^{*}$ is explicitly included in the objective function. We next investigate two key questions: in what form should contact prior guide contact planning, and how should a feasible approach and contact trajectory be defined mathematically?

1) Role of Contact Guidance: In the previous section, we discussed how CIMPC methods suffer from gradient sparsity and proposed CSO, which serves as a search over the finite sampled candidate contact set $\mathcal{I}_{\text{avail}}$ on the object surface and provide contact guidance. But this raises another open question: in what form should the guidance affect contact execution? The most straightforward approach is full coupling, where the contact planning strictly follows and executes the guidance from the CSO. However, such a fully coupled scheme may compromise control robustness because the upper-level surrogate abstracts away detailed contact interactions. The object-centric search performed by CSO does not explicitly account for robot morphology, resulting in an inherent gap between contact selection and execution. Moreover, contact-rich manipulation is inherently underdetermined, as numerous contact-wrench and contact-location trajectories can accomplish the same task. Strictly executing the CSO-optimal trajectory would therefore restrict this intrinsic solution diversity in robot–object interaction. To address this limitation, we introduce an objective-based contact planning module named CPO. Through the ranking strategy, the CPO balances the CSO-optimal contact against nearby and other candidate contacts, establishing a loose coupling between high-level contact selection and low-level contact planning. This design benefits from sampled-set guidance to escape local optima while preserving the flexibility of contact planning for execution.

2) Contact Switching Trajectory: Existing CIMPC methods mainly induce nearby contacts through attractive costs, without explicitly modeling the detouring motions required for either approaching designated contacts or switching between contacts. To incorporate the upper-level guidance, we define the approaching trajectory in the contact-free space $\mathcal{S}_{\text{free}}$ as:

- All intermediate waypoints lie in the collision-free space.
- The terminal waypoint establishes contact, and the contact force lies within the friction cone.

As shown in Fig. 5, the robot–object interaction states form a low-dimensional manifold embedded in the full system state space. A detouring motion can be viewed as planning a kinodynamically feasible bridging manifold $\mathcal{S}_{\text{approach}}$ within the contact-free space $\mathcal{S}_{\text{free}}$, connecting two states on the contact manifold $\mathcal{S}_{\text{contact}}$. This is challenging for CIMPC methods because, as discussed above, $\mathcal{S}_{\text{free}}$ provides little informative contact-induced gradient for such transitions. To naturally bridge the approaching, contact, and contact-switching phases without relying on a discrete state machine, we formulate CPO

under a unified objective that jointly governs contact execution and pre-contact detouring and approach motions.

B. Ranking Strategy of Contact Locations

In addition to the optimal contact location $\hat{\boldsymbol{p}}_{\mathrm{r}}^{*}$ from the CSO prediction, we always consider the nearest point $\boldsymbol{p}_{\mathrm{near}}$ to the current robot configuration as a candidate contact location:

$$\boldsymbol{p}_{\mathrm{near}} = \arg \min_{\boldsymbol{p}_i \in \mathcal{I}_{\mathrm{avail}}} \left\| \boldsymbol{p}_{\mathrm{ee}} - \boldsymbol{p}_i^{\mathrm{ee}} \right\|_2, \tag{33}$$

where $\boldsymbol{p}_i^{\mathrm{ee}}$ is the end-effector position corresponding to contact at $\boldsymbol{p}_i$. We reformulate the problem (32) so that the placing term tracks the nearby contact as the reference contact location, $\boldsymbol{p}_{\mathrm{ref}} = \boldsymbol{p}_{\mathrm{near}}$, while the lifting term detours toward $\hat{\boldsymbol{p}}_{\mathrm{r}}^{*}$; the ranking strategy weights the two terms in real time. Accordingly, the ranking output determines whether to maintain the current contact or detach and switch to $\hat{\boldsymbol{p}}_{\mathrm{r}}^{*}$ in the contact phase, and whether to establish a nearby contact or detour toward $\hat{\boldsymbol{p}}_{\mathrm{r}}^{*}$ in the pre-contact phase. We next define the evaluation criteria for the ranking strategy.

For each candidate contact location $\boldsymbol{p}_i$, the optimal cost reduction obtained from (29) is denoted as $\Delta\ell_t^i = \ell(\boldsymbol{x}_t) - \ell(\hat{\boldsymbol{x}}_t^{+})$. Based on the predicted cost reduction range of all candidate contact locations, $\left[\Delta\ell_t^{\min}, \Delta\ell_t^{\max}\right]$, the relative ranking score of the nearby contact $\boldsymbol{p}_{\mathrm{near}}$ is defined as

$$\rho_t = \Pi_{[0,1]}\left(\frac{\Delta\ell_t^{\mathrm{near}} - \Delta\ell_t^{\min}}{\max(\Delta\ell_t^{\max} - \Delta\ell_t^{\min}, \epsilon)} \right), \tag{34}$$

where the fraction performs min-max normalization and $\Pi_{[0,1]}$ projects the resulting score onto $[0,1]$. The ranking score ρ_t indicates the relative improvement of the nearby contact compared with all candidate contacts.

We use $\kappa_t \in \{0,1\}$ to decide whether the nearby contact replaces the CSO-selected contact:

$$\kappa_t = \mathbf{1}(\rho_t \geq \bar{\rho}), \tag{35}$$

where $\bar{\rho}$ is a predefined ranking threshold and $\mathbf{1}(\cdot)$ denotes the indicator function. When $\kappa_t = 1$, the nearby contact $\boldsymbol{p}_{\mathrm{near}}$ provides a sufficiently competitive cost reduction and can serve as the reference contact. Otherwise, the reference remains the CSO-selected contact $\hat{\boldsymbol{p}}_{\mathrm{r}}^{*}$. The trigger variable κ_t therefore evaluates the relative quality of the nearby contact among the sampled candidates and determines whether contact relocation is allowed. To avoid frequent switching due to transient ranking variations, we apply exponential moving average (EMA) smoothing to κ_t:

$$\bar{\kappa}_{t+1} = (1-\beta)\bar{\kappa}_t + \beta\kappa_t, \qquad \gamma_{t+1} = \mathbf{1}\left(\bar{\kappa}_{t+1} \geq \tfrac{1}{2}\right), \tag{36}$$

where $\beta \in [0,1]$ is the smoothing coefficient, $\bar{\kappa}_t \in [0,1]$ is the EMA-filtered trigger with $\bar{\kappa}_0 = 0$, and $\gamma_t \in \{0,1\}$ is the resulting binary phase variable used in the CPO objective. Here $\gamma_t = 0$ selects lifting toward the CSO-selected contact, whereas $\gamma_t = 1$ selects placing toward the nearby contact.

The overall ranking process repeatedly evaluates the nearby contact under the current configuration. CPO selects the nearby contact $\boldsymbol{p}_{\mathrm{near}}$ when $\gamma_t = 1$ and the guidance $\hat{\boldsymbol{p}}_{\mathrm{r}}^{*}$ generated by CSO when $\gamma_t = 0$; the filtered trigger $\bar{\kappa}_t$ remains continuous, while the phase is selected by its $1/2$ threshold. The complete procedure is summarized in Algorithm 2. This hierarchical design establishes a weak coupling between CSO and CPO, allowing CPO to exploit the guidance from CSO while retaining the flexibility to autonomously determine executable contact strategies.

Algorithm 2 Ranking Strategy of CPO

Input: $\hat{\boldsymbol{p}}_r^{*}$ predicted by CSO, current robot configuration $\boldsymbol{q}_k$, threshold $\bar{\rho}$, EMA coefficient β, and current filter state $\bar{\kappa}_t$
Output: $\boldsymbol{p}_{\mathrm{ref}}$ for the placing term and phase cost ℓ_{lp}

```
Compute nearest point p_near to end-effector via (33)
Evaluate Δℓ_t^near and [Δℓ_t^min, Δℓ_t^max] via (29)
Compute ρ_t and κ_t via (34) and (35)
Update κ̄_{t+1} and γ_{t+1} = 1(κ̄_{t+1} ≥ 1/2) via (36)
Set p_ref ← p_near for ℓ_place; ℓ_lift continues to target p̂_r^*
Compute ℓ_lp = (1 − γ_{t+1})ℓ_lift + γ_{t+1}ℓ_place via (39), with
    γ_{t+1} ∈ {0, 1} selecting lifting or placing
return p_ref, ℓ_lp
```

C. Formulation of CPO

Contact planning inherently consists of multiple phases, including approaching, contact establishment, and contact switching. However, the CIMPC formulation does not naturally support contact switching, because contact switching requires leaving $\mathcal{S}_{\mathrm{contact}}$ and traversing the contact-free space $\mathcal{S}_{\mathrm{free}}$, where the CIMPC objective provides no effective gradient information for trajectory optimization. To address this issue, we introduce a novel objective formulation that enables multi phases planning within a unified framework. The proposed formulation is general to different task settings and compatible with various optimization solvers.

As illustrated in Fig. 6, the trajectory optimization is decomposed into three stages. First, the end-effector leaves the current contact and moves around the object toward an intermediate waypoint $\boldsymbol{p}_{\mathrm{lift}}$ or pose $\boldsymbol{q}_{\mathrm{lift}}$. This intermediate target guides the robot toward a pre-contact configuration above the object. Second, the end-effector is guided toward $\boldsymbol{p}_{\mathrm{ref}}$ to establish contact. This design is inspired by footstep planning in legged robots [39]. The corresponding costs are:

1) Lifting Phase: During the lifting phase, when the nearest contact point $\boldsymbol{p}_{\mathrm{near}}$ is not accepted by the ranking strategy ($\gamma_t = 0$), a hybrid potential field guides the robot to detach from the object and move above the guidance. It consists of a repulsive term from the object:

$$\ell_{\mathrm{obs}} = \begin{cases} -\log\left(\|\boldsymbol{p}_{\mathrm{ee}} - \boldsymbol{p}_{\mathrm{o}}\|^2 + \varepsilon\right), & \text{if } \|\boldsymbol{p}_{\mathrm{ee}} - \boldsymbol{p}_{\mathrm{o}}\|^2 < \sigma, \\ 0, & \text{otherwise.} \end{cases} \tag{37}$$

and an attractive term toward the lifting point $\boldsymbol{p}_{\mathrm{lift}}$ or lifting configuration $\boldsymbol{q}_{\mathrm{lift}}$. The lifting target is a moving waypoint on a lift–cross–drop path that rises above the object, crosses to the CSO-selected contact $\hat{\boldsymbol{p}}_{\mathrm{r}}^{*}$, and descends to it. The waypoint follows the current path segment, and the path is latched during crossing to prevent ranking fluctuations from reversing the motion. We use $\ell_{\mathrm{att}} = \|\boldsymbol{p}_{\mathrm{ee}} - \boldsymbol{p}_{\mathrm{lift}}\|^2$ or $\ell_{\mathrm{att}} = \|\boldsymbol{q}_{\mathrm{ee}} - \boldsymbol{q}_{\mathrm{lift}}\|^2$, and define $\ell_{\mathrm{lift}} = w_{\mathrm{att}}\ell_{\mathrm{att}} + w_{\mathrm{obs}}\ell_{\mathrm{obs}}$. This potential formulation

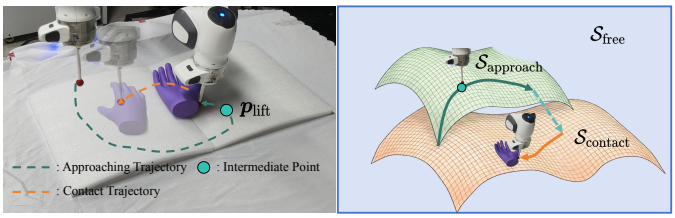


Fig. 5. (a) Illustrative example of a feasible approach and contact trajectory. (b) Explanation of CPO in a geometric perspective. CPO can be characterized as operating over two manifolds: contact states in $\mathcal{S}_{\text{contact}}$ and approaching trajectories on the manifold $\mathcal{S}_{\text{approach}}$ embedded in the contact-free state space $\mathcal{S}_{\text{free}}$. CPO must not only generate contact trajectories on $\mathcal{S}_{\text{contact}}$ but also address the gradient-sparse issue and generate trajectories on $\mathcal{S}_{\text{approach}}$.

encodes the lifting behavior and avoids optimization failure caused by gradient-sparse regions during contact breaking.

2) Placing Phase: During the placing phase, when the robot reaches a position above $\hat{\boldsymbol{p}}_r^*$, or when $\boldsymbol{p}_{\text{near}}$ is accepted by the ranking strategy ($\gamma_t = 1$), the end-effector is guided to establish contact with the object using $\ell_{\text{place}} = w_{\text{con}}\ell_{\text{con}}$. The cost attracts the end-effector to an inward offset of $\boldsymbol{p}_{\text{ref}}$.

$$\ell_{\text{con}} = \|\boldsymbol{p}_{\text{ee}} - (\boldsymbol{p}_{\text{ref}} - \delta \boldsymbol{n}_{\text{ref}})\|^2 . \tag{38}$$

Here $\boldsymbol{n}_{\text{ref}}$ is the outward unit surface normal at $\boldsymbol{p}_{\text{ref}}$, and the inward offset is fixed at $\delta = 0.02\,\text{m}$ (2 cm). Attracting the end-effector directly to $\boldsymbol{p}_{\text{ref}}$ may fail to establish physical contact when the object mesh or contact estimation is inaccurate, since perception or projection errors can cause $\hat{\boldsymbol{p}}_r^*$ to deviate from the actual object surface. The inward offset improves robustness against such errors.

3) Switching Parameter: The switching between lifting and placing behaviors is determined by the binary phase variable $\gamma_t \in \{0, 1\}$ in (36), obtained by thresholding the EMA-filtered trigger $\bar{\kappa}_t$. The overall lift-and-place cost is:

$$\ell_{\text{lp}} = (1-\gamma)\ell_{\text{lift}} + \gamma\ell_{\text{place}}, \tag{39}$$

Since γ is binary and held fixed during each CPO solve, one phase cost is active at a time, consistently with Algorithm 2. where ℓ_{lp} is incorporated into the stage cost ℓ_{cpo} in (32). By defining the terminal cost $V_f(\boldsymbol{x}_N) = w_{\text{cso}}\ell_{\text{cso}}(\boldsymbol{x}_N)$ on the predicted terminal state, the CPO formulation is obtained as:

$$\begin{aligned} \min_{\boldsymbol{u}_{0:N-1}} \quad & \textstyle\sum_{k=0}^{N-1} \left(w_{\text{lp}}\ell_{\text{lp}}(\boldsymbol{x}_k, \boldsymbol{q}_k, \boldsymbol{u}_k, \boldsymbol{p}_{\text{ref}}) + w_u\|\boldsymbol{u}_k\|^2\right) + V_f(\boldsymbol{x}_N) \\ \text{s.t.} \quad & \boldsymbol{x}_{k+1} = f_{\text{cf}}(\boldsymbol{x}_k, \boldsymbol{u}_k), \quad k = 0, \ldots, N-1, \\ & \boldsymbol{x}_k \in \mathcal{X}, \quad k = 0, \ldots, N, \\ & \boldsymbol{u}_k \in \mathcal{U}, \quad k = 0, \ldots, N-1. \end{aligned} \tag{40}$$

The formulation in (40) retains the CIMPC objective and supports hybrid contact trajectories.

D. Solvers of CPO

Since CPO specifies the optimization formulation rather than a specific solver, various reactive motion planners can be adopted to generate contact trajectories. Below, we present two solvers employed in our experiments.

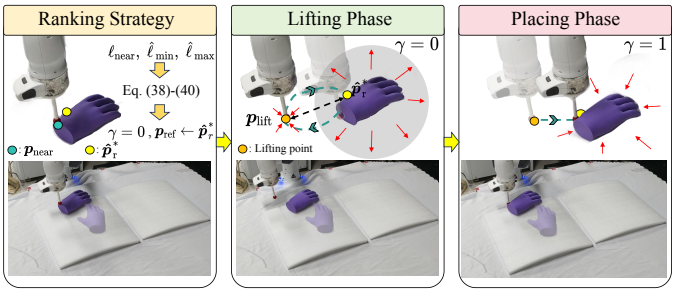


Fig. 6. Illustration of the CPO-based contact planning module. (a) Ranking strategy. The evaluation runs continuously in real time; its binary outcome κ_t is filtered into $\bar{\kappa}_t$ and thresholded at $1/2$ to obtain the phase variable γ_t. (b) Lifting phase ($\gamma_t = 0$): $\boldsymbol{p}_{\text{near}}$ is not sufficiently competitive, and the potential field guides the robot to $\boldsymbol{p}_{\text{lift}}$ toward $\hat{\boldsymbol{p}}_r^*$. (c) Placing phase ($\gamma_t = 1$): once above $\hat{\boldsymbol{p}}_r^*$ or when $\boldsymbol{p}_{\text{near}}$ is accepted by the ranking strategy, an attractive potential guides the robot to contact with the object.

1) Operational-Space Control: We simplify the full-order contact planning problem to an end-effector point contact planning problem, avoiding high-DoF robot trajectory optimization. The resulting joint torques are generated by operational space control (OSC) [40] to track the CPO output. Specifically, the state in (40) is reduced to $\boldsymbol{s} \in \mathbb{R}^{10}$, consisting of the object position, object orientation represented by a quaternion, and the end-effector position $\boldsymbol{p}_{\text{ee}} \in \mathbb{R}^3$. Solving (40) yields the control input $\boldsymbol{u} = \Delta\boldsymbol{p}_{\text{ee}}$.

Given the desired pose $(\boldsymbol{p}_d, \boldsymbol{R}_d)$ with $\boldsymbol{p}_d = \boldsymbol{p}_{\text{ee}} + \boldsymbol{u}$ and a preset $\boldsymbol{R}_d$, the position and orientation errors are computed as $\boldsymbol{e}_{\text{pos}} = \boldsymbol{u}$, and $\boldsymbol{e}_{\text{R}} = \boldsymbol{R}_{\text{ee}}^\top \boldsymbol{R}_d$. Converting $\boldsymbol{e}_{\text{R}}$ to quaternion form gives $\boldsymbol{e}_{\text{quat}} = \left[\boldsymbol{\epsilon}_e, \eta_e\right]^\top \in \mathbb{R}^4$, and after sign correction,

$$\boldsymbol{e}_{\text{quat}} \leftarrow \begin{cases} \boldsymbol{e}_{\text{quat}}, & \eta_e \geq 0, \\ -\boldsymbol{e}_{\text{quat}}, & \eta_e < 0, \end{cases} \qquad \boldsymbol{e}_{\text{rot}} = -\boldsymbol{R}\boldsymbol{\epsilon}_e. \tag{41}$$

By defining $\boldsymbol{e}_{\text{pose}} = \left[\boldsymbol{e}_{\text{pos}}, \boldsymbol{e}_{\text{rot}}\right]^\top$, the task-space and null-space torques are given by:

$$\boldsymbol{\tau}_{\text{task}} = \boldsymbol{J}^\top(\boldsymbol{q})\left(-\boldsymbol{K}_x\boldsymbol{e}_{\text{pose}} - \boldsymbol{D}_x\dot{\boldsymbol{x}}\right), \tag{42}$$

$$\boldsymbol{\tau}_{\text{null}} = \left(\boldsymbol{I} - \boldsymbol{J}^\top(\boldsymbol{J}^\top)^\dagger\right)\left(\boldsymbol{K}_q(\boldsymbol{q}_{\text{init}} - \boldsymbol{q}) - 2\sqrt{\boldsymbol{K}_q}\,\dot{\boldsymbol{q}}\right), \tag{43}$$

and the final control torque is $\boldsymbol{\tau} = \boldsymbol{\tau}_{\text{task}} + \boldsymbol{\tau}_{\text{null}} + g(\boldsymbol{q})$. This controller is used in all simulations and real-world experiments in Section V, and is suitable for tasks such as tabletop manipulation, where high orientation diversity is not critical.

2) Sampling-based Motion Planner: Another formulation is to directly solve the full-order planning problem of the robot. In this case, the state in (40) is given by $\boldsymbol{x} \in \mathbb{R}^{7+n_{\text{DoF}}}$, where n_{DoF} denotes the number of robot DoFs. For this class of problems, we employ sampling-based MPC methods as the solver, such as MPPI [41]. Recent studies have demonstrated the impressive performance of MPPI in solving high-dimensional optimization problems for redundant robots, enabling real-time computation while avoiding issues related to gradient-based methods. The cost function is defined in the same manner as in the previous formulation, except that $\boldsymbol{p}_{\text{ee}}$ is computed via forward kinematics for the full-order problem.

V. Experiment: Nonprehensile Manipulation

This section evaluates the SCSP framework through nonprehensile manipulation experiments, which represent a class of contact-rich manipulation tasks characterized by frequent contact-state transitions and unstable control. The tasks consist of on-ground rotation and on-ground flipping. On-ground rotation requires the robot to move an object to a target pose on the ground through 2D pushing and rotating, whereas on-ground flipping involves 6D manipulation. We select a set of irregularly shaped objects from the ContactDB dataset [42] for validation. The objects are initialized at random poses with $x_{\text{init}}, y_{\text{init}} \in [-0.3, 0.3]$ m, and Euler angles $(\phi_{\text{init}}, \theta_{\text{init}}, \varphi_{\text{init}})$ uniformly sampled from $[-\pi, \pi]$ rad. We define the conditions of task success as:

$$\|\boldsymbol{p}_o - \boldsymbol{p}_{\text{goal}}\| \le 0.02\,[\text{m}],\ 1 - (\boldsymbol{\psi}_{\text{goal}}^\top \boldsymbol{\psi}_o)^2 \le 0.015, \quad (44)$$

where the orientation tolerance corresponding to the quaternion error threshold is $14.1°$. A trial is considered successful if the conditions are satisfied within 2500 rollout steps.

Simulations are conducted in `MuJoCo` [43] and `IsaacGym` [44] with a fixed integration time step $h = 0.02$s. For the sampling method in our CSO module, we use `CoACD` [45] for convex decomposition of the objects and `trimesh` for mesh processing and local frame construction. CSO is solved using GPU-parallel CEM, while CPO is solved using `acados` SQP-RTI with HPIPM and a solver tolerance of 10^{-4}. Unless otherwise stated, experiments are conducted on a desktop computer with an Intel i7-13620H processor, 32 GB RAM, and an NVIDIA GeForce RTX 5090 GPU.

We applied randomization to the environment and object parameters. The friction coefficients of the object μ_{o}, fingertip μ_{r}, and ground μ_{env} are all set to 0.5 with random perturbations of ± 0.2. The object mass is randomly sampled from the discrete set $\{0.01, 0.1, 1.0, 10.0\}$ kg with equal probability for each trial, while SCM uses a fixed nominal mass of 0.1 kg. We set $\boldsymbol{M}_o = \text{diag}(50, 50, 50, 0.05, 0.05, 0.05)$ and $\boldsymbol{K}_r = 300\boldsymbol{I}_7$, and the dynamic parameters used in SCM are not measured precisely, but are roughly set to their mean values.

For CSO, the default configuration samples $n_s = 70$ candidate contacts, retains the two lowest-cost candidates, and uses a one-step prediction ($T = 1$) with 20 GPU-parallel CEM iterations. CSO and CPO share a prediction step of $0.02\,\text{s}$ and pose weights $w_{\text{pos}} = 500$ and $w_{\text{quat}} = 20$. CSO uses force and tangential-force regularization weights of 0.01 and 0.03, respectively, with a $3\,\text{N}$ force-command limit corresponding to $\hat{\lambda}^n_{\max} \approx 1.59\,\text{N}$ under the configured friction pyramid. CPO uses $N = 5$, componentwise control bounds $|u_j| \le 0.005\,\text{m}$, and weights $w_{\text{lp}} = 1$, $w_u = 50$, and $w_{\text{cso}} = 10$. Its lifting and reference-contact attraction weights are both 20, while the repulsive-field term is disabled. The lift–cross–drop path uses clearance margins $d_{\text{top}} = 0.025\,\text{m}$ and $d_{\text{ref}} = 0.015\,\text{m}$ and a per-cycle step bound of $0.005\,\text{m}$ on $\boldsymbol{p}_{\text{lift}}$. The bounding box defining z_{top} is the axis-aligned bounding box of the object mesh in its body frame, with center $\boldsymbol{c}$ and half-extents $\boldsymbol{e}$; it is rotated with the estimated object pose and $z_{\text{top}} = (\boldsymbol{p}_o + \boldsymbol{R}_o\boldsymbol{c})_z + \sum_i |R_{o,zi}|\, e_i$ is the top of the resulting oriented box, so z_{c} follows the object's current orientation. Segment switching uses tolerances of $0.004\,\text{m}$ on the height z_{c} and $0.008\,\text{m}$ on the horizontal distance to the waypoint; the latched waypoint is released when the end-effector is within $0.012\,\text{m}$ of the target or the object moves by more than $0.015\,\text{m}$. During the flipping stage, the orientation error measures alignment of the object's target upright axis with the world vertical, leaving yaw unconstrained. Let $\boldsymbol{a}_g = \boldsymbol{R}(\boldsymbol{x}_{\text{goal}})^\top e_z$, where $e_z = [0, 0, 1]^\top$. The normalized upright error is $\ell_{\text{up}}(\boldsymbol{x}_o) = \frac{1}{2}\left[1 - e_z^\top \boldsymbol{R}(\boldsymbol{x}_o)\boldsymbol{a}_g\right]$. The flipping-stage object cost is $\ell_{\text{cso}} = 500\ell_{\text{pos}} + 20\ell_{\text{up}}$, i.e., ℓ_{pose} in (7) with the quaternion-error term replaced by this yaw-free upright error. Here $\ell_{\text{pos}} = \|\boldsymbol{p}_o - \boldsymbol{p}_{\text{goal}}\|^2$, except for the Stanford bunny, which uses $\ell_{\text{pos}} = \max(\|\boldsymbol{p}_{o,xy} - \boldsymbol{p}_{\text{goal},xy}\| - 0.08\,\text{m}, 0)^2$ to allow vertical motion and planar displacement within an $8\,\text{cm}$ radius during flipping.

From Table I, we select methods that are applicable to 6D manipulation and can manipulate objects with arbitrary geometries, rather than being restricted to regular shapes, as baselines, including:

- **DyWA** [17]: An end-to-end 6D nonprehensile policy evaluated on unseen object geometries using the released single-view student policy in `IsaacGym`.
- **ContactSDF** [24]: A differentiable multi-contact model based on signed distance functions (SDFs), which provides closed-form collision detection and time-stepping prediction for contact-rich manipulation.
- **STOCS-3D** [23]: An offline method that jointly optimizes contact selection and a contact-implicit trajectory optimization (CITO) problem for 3D manipulation. We use STOCS-3D to generate the CITO trajectory offline and execute the resulting trajectory with operational-space control (OSC).
- **Sampling** [25]: A hierarchical method that samples candidate contacts and evaluates them with a lower-level CIMPC controller. We use the released implementation with its original C3 contact model, Gurobi MIQP solver, and contact-selection method.
- **CF-MPC** [22]: A complementarity-free MPC method with a closed-form contact model and limited explicit contact-selection capability.

A. Robot Manipulation in Simulation

We evaluate SCSP in redundant robot manipulation tasks. Compared with a floating end-effector, contact problems for redundant robots introduce complex kinematic constraints. In the experiments, we demonstrate the superiority of SCSP and validate the soundness of our framework.

Due to the strong dependencies of the source code released for DyWA, which are difficult to transfer, we follow its setup and conduct all experiments in `IsaacGym` for consistency, with the same simulation settings described above. All methods use our link-mounted sphere as the end-effector to replicate point contacts. Since DyWA uses the Franka gripper, we adjust the link length so that the sphere's center coincides with the gripper's center. The Cartesian stiffness $\boldsymbol{K}_x = \text{diag}(1000, 1000, 1000, 50, 50, 50)$ and $\boldsymbol{D}_x = \sqrt{2\boldsymbol{K}_x}$. All methods use the same settings as above, with the control

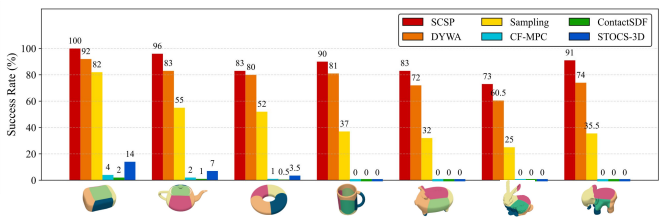


Fig. 7. Experimental results of robot manipulation in simulation. SCSP significantly outperforms other model-based methods and exceeds the performance of well-trained learning-based methods.

dimension set by each method's formulation. $\boldsymbol{p}_{\text{ee}}$ is obtained via differentiable forward kinematics.

The experimental results are shown in Fig. 7, Fig. 8, and Table II. SCSP achieves a higher success rate than other model-based algorithms and outperforms DyWA on all objects except the torus. Additionally, SCSP achieved the shortest execution time among the comparison methods (the shorter times of A2 and CF-MPC are averaged over very few successful trials), demonstrating that SCSP can generate correct contact locations through CSO, thereby efficiently minimizing the pose cost of the object. SCSP also achieved the smallest convergence position error and quaternion error among the comparison methods, while A4 and A5 attain comparable errors at a much higher planning cost. Furthermore, we observed that during experiments, DyWA tends to flip objects by pressing the edges of the top surface, whereas our method makes more comprehensive decisions regarding whether to lift the object by pushing the bottom or by pressing the top. Although many flipping tasks can be accomplished through pressing, for certain objects (such as elongated objects), the pressing approach requires applying substantial torque to the object and demands sufficiently high coefficients of friction. Based on our observations, the global contact selection mechanism in CSO enables SCSP to exhibit greater diversity in its manipulation behaviors compared with other methods, rather than converging to a single strategy. Fig. 9 shows the actual CSO cost evaluation and contact selection during flipping: the executed contact switches between $\boldsymbol{p}_{\text{near}}$ and $\hat{\boldsymbol{p}}_r^*$, i.e., the ranking strategy does not always select either one.

We further validated the impact of different numbers of sampled candidate points on performance. As shown in Table III, when the number of sampled points reaches 70, nearly the highest success rate is achieved with relatively low SCSP computation time. As the number of sampled points continues to increase, the success rate plateaus and computational efficiency deteriorates. This indicates that increasing the number of sampled points does not necessarily improve performance. There exists an optimal convergence value. For nonprehensile manipulation tasks, 70 to 100 sampled points are sufficient to characterize the geometric features of object geometries.

B. Ablation Studies

We ablate the ranking strategy, CSO, and CPO under the same experimental setup as in Section V-A to demonstrate their contributions. We also examine contact-model fidelity and the feasibility of CSO-generated contact trajectories.

1) Functional Analysis of SCSP Components: **Ranking strategy (A1–A2):** CPO strictly follows CSO-optimal guidance p_r^* (A1) or always executes the nearest contact p_{near} (A2).

CSO (A3–A5): A3 replaces SCM with a coarse contact model that encloses the object in an oriented bounding box (OBB), ignores object–environment contact, and evaluates candidate contact locations solely based on the object pose error. This variant assesses the model fidelity needed to provide useful contact priors. A4 replaces SCM with MuJoCo's QP-based contact model; A5 directly applies CMA-ES to solve (14) and obtain $(\boldsymbol{p}_{\text{r}}^*, \boldsymbol{\lambda}_{\text{r}}^*)$, without sampling, constructing, or evaluating a candidate contact set. The SDF-distance constraint $d_{\text{SDF}}(\boldsymbol{p}_{\text{r}}) = 0$ enforces that every optimized contact point lies on the object surface.

CPO (A6–A7): A6 removes the phase-dependent attraction and repulsion potentials and directly optimizes motion trajectories; A7 removes contact guidance from CPO and solves the resulting problem with MPPI [41]: the ranking strategy and the lifting phase are removed, and the objective in (40) retains only the attraction toward $\boldsymbol{p}_{\text{near}}$ in (38) and the object cost ℓ_{cso}, with the same cost weights and horizon as CPO. MPPI samples end-effector displacements under the same f_{cf} and control bounds as CPO, with $N = 5$, 128 samples, one iteration, noise $\sigma = 0.003\,\text{m}$, and temperature 0.005. Table II summarizes the results.

A1 and A3 substantially outperform A2, demonstrating the importance of contact guidance over purely local contact selection. However, A1 switches contact points too frequently, causing some tasks to fail. Even the coarse contact model in A3 yields substantial performance gains, suggesting that useful contact priors do not require detailed contact modeling. A4 and A5 show that higher model fidelity or more refined optimization does not necessarily improve task success. Compared with CPO's cost formulation, directly solving the full trajectory planning problem in A6 imposes a greater computational burden, reducing success rates and substantially increasing planning time. Despite using MPPI for stronger exploration of the same CPO problem without contact guidance, A7 still achieves low success rates, indicating that increased exploration alone cannot effectively replace contact guidance.

We also test sensitivity to the acceptance threshold $\bar{\rho}$ and the EMA coefficient β (Fig. 10). Setting $\bar{\rho} = 0$ accepts nearby contacts without filtering their quality, recovering A2, whereas $\bar{\rho} = 1$ restricts acceptance to optimal contacts, corresponding to A1. Smaller $\bar{\rho}$ values make nearby contacts easier to accept. Setting $\beta = 1$ disables smoothing and permits abrupt changes between consecutive decisions, whereas smaller positive values provide stronger smoothing but slow the response; $\beta = 0$ prevents the filtered trigger $\bar{\kappa}_t$ from updating. The sweep results support the default settings $\bar{\rho} = 0.7$ and $\beta = 0.2$, which are used in the experiments below.

2) Contact Model Evaluation: We evaluate contact-model fidelity and the feasibility of the resulting ideal contact trajectories. We report the linear- and angular-velocity errors between SCM and the corresponding MuJoCo and IsaacGym references under contact transition, sliding, sticking, and rotation/flipping rollouts. The contact state of a sample is determined from the loaded contacts of the reference input state:

TABLE II
COMPARISON AND ABLATION RESULTS FOR NONPREHENSILE MANIPULATION TASKS IN THE ROBOT EXPERIMENTS

Task	Metric	Comparison						Ablation						
		SCSP*	DyWA	ContactSDF	STOCS-3D	Sampling	CF-MPC	A1	A2	A3	A4	A5	A6	A7
Rotation	Success Rate	**0.95**	0.86	0.01	0.07	0.52	0.01	0.37	0.06	0.57	0.66	0.89	0.00	0.17
	Execution Time (s)	15.16 ± 11.82	18.12 ± 18.46	–	–	18.97 ± 43.91	13.18 ± 8.67	25.31 ± 11.03	12.74 ± 0.14	17.91 ± 9.15	20.77 ± 9.82	19.06 ± 8.53	–	16.21 ± 1.90
	Planning Time (ms)	21.12 ± 11.31	31.00 ± 7.67	22.30 ± 11.17	$(5.93 \pm 5.30) \times 10^5$	108.12	22.32	26.55 ± 20.42	12.91 ± 2.52	21.21 ± 7.43	99.94 ± 61.10	612.53 ± 168.13	81.38 ± 11.62	12.09 ± 2.23
	Obj. Pos. Error (m)	0.05 ± 0.11	0.07 ± 0.13	0.06 ± 0.03	0.17 ± 0.07	0.08 ± 0.07	0.20 ± 0.05	0.11 ± 0.13	0.17 ± 0.08	0.09 ± 0.09	0.04 ± 0.06	0.06 ± 0.14	0.29 ± 0.13	0.29 ± 0.16
	Obj. Quat. Error	0.09 ± 0.22	0.12 ± 0.29	0.64 ± 0.34	0.58 ± 0.37	0.30 ± 0.37	0.56 ± 0.19	0.19 ± 0.30	0.31 ± 0.37	0.18 ± 0.32	0.07 ± 0.19	0.07 ± 0.20	0.58 ± 0.33	0.30 ± 0.37
Flipping	Success Rate	**0.81**	0.69	0.00	0.00	0.39	0.01	0.19	0.00	0.37	0.26	0.60	0.00	0.00
	Execution Time (s)	18.75 ± 9.79	20.56 ± 12.96	–	–	22.68 ± 12.95	14.56 ± 6.37	22.05 ± 9.36	–	23.60 ± 11.86	22.71 ± 12.59	24.23 ± 11.50	–	–
	Planning Time (ms)	20.91 ± 8.03	31.00 ± 7.67	23.22 ± 12.22	$(2.39 \pm 1.25) \times 10^5$	102.23 ± 29.72	81.00	24.94 ± 20.23	12.59 ± 2.34	21.80 ± 11.85	132.95 ± 49.02	577.81 ± 113.82	81.75 ± 10.63	12.86 ± 2.90
	Obj. Pos. Error (m)	0.04 ± 0.05	0.13 ± 0.17	0.20 ± 0.09	0.17 ± 0.04	0.10 ± 0.05	0.19 ± 0.05	0.16 ± 0.14	0.15 ± 0.05	0.12 ± 0.12	0.09 ± 0.09	0.11 ± 0.15	0.24 ± 0.08	0.35 ± 0.16
	Obj. Quat. Error	0.12 ± 0.27	0.26 ± 0.39	0.87 ± 0.19	0.79 ± 0.25	0.40 ± 0.38	0.56 ± 0.19	0.38 ± 0.41	0.73 ± 0.32	0.47 ± 0.41	0.47 ± 0.41	0.29 ± 0.40	0.76 ± 0.30	0.80 ± 0.25

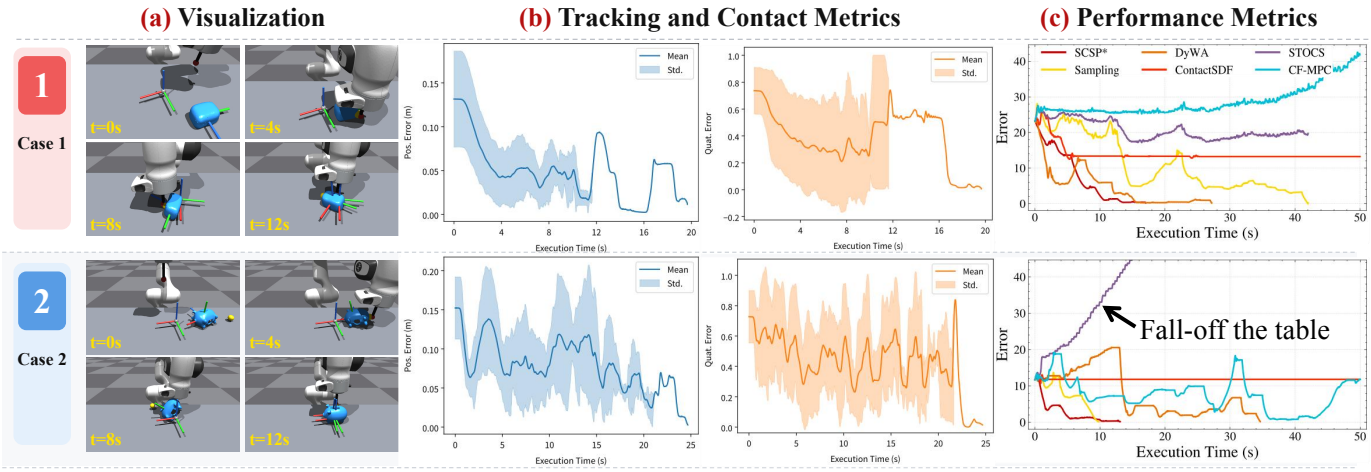


Fig. 8. Nonprehensile manipulation experiments in simulation. (a): Snapshots of SCSP in the robot experiments, where the goal pose is visualized either transparently or as a floating coordinate frame. (b): The mean and standard deviation curves of the position and orientation errors over steps across 10 SCSP trials. (c) The pose error ℓ_{pose} of different methods during manipulation. The comparison methods either require longer execution time or fail due to local minima or solver failures.

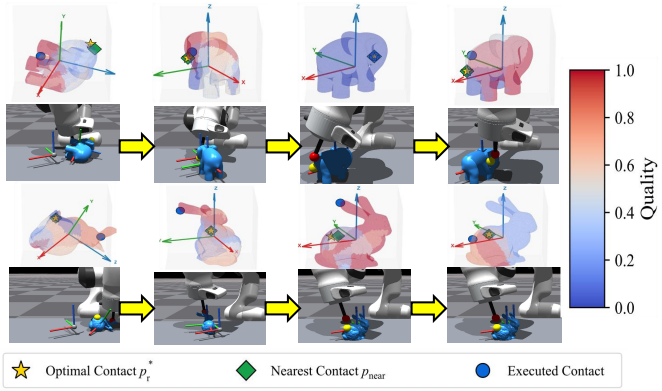


Fig. 9. Visualization of the contact map during the elephant and bunny flipping tasks. Red indicates a larger cost reduction predicted by CSO and thus higher contact quality.

TABLE III
EVALUATION OF THE EFFECT OF THE NUMBER OF CANDIDATE POINTS

Sampling numbers in CSO	**5**	**10**	**70**	**120**	**500**
Avg. Success Rate	0.25	0.59	**0.88**	**0.88**	0.85
SCSP Computation Time (s)	**0.012**	0.017	0.025	0.032	0.089

contact transition if any loaded contact has a normal relative speed above $0.01\,\mathrm{m/s}$ (impact or lift-off); otherwise *sliding* if any loaded contact slips faster than $1\,\mathrm{mm/s}$; otherwise

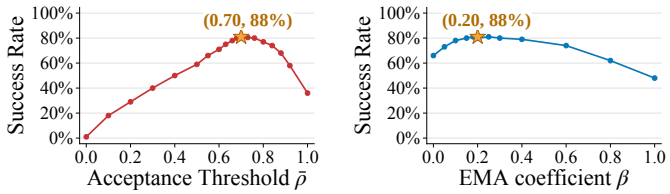


Fig. 10. Sensitivity of the measured success rate to the acceptance threshold $\bar{\rho}$ and EMA coefficient β. Each parameter is varied independently while the others are fixed at their default values.

sticking. The three states partition all contact situations; the transition samples comprise impact and lift-off initial states (32 and 16 per object), and airborne states are excluded. For the contact-mode measurements, samples are further grouped by the reference active-contact count $n_c \in \{1, 2, \geq 3\}$. The benchmark uses seven objects, 32 initial states per object and contact state, and 10 rollout windows per rollout state. In the rollout columns, every engine is reset to the MuJoCo reference state at each of the 250 steps of a window and advanced once, so all columns report one-step errors from a common reference state and differ only in the state distribution of the rotation and flipping manoeuvres. The larger IsaacGym transition errors and the spread of its rollout columns come from the depenetration response of PhysX when it is initialized at the slightly penetrating states produced by MuJoCo's soft contacts. Table IV shows that, apart from this state, the linear errors stay at the centimetre-per-second level and the angular

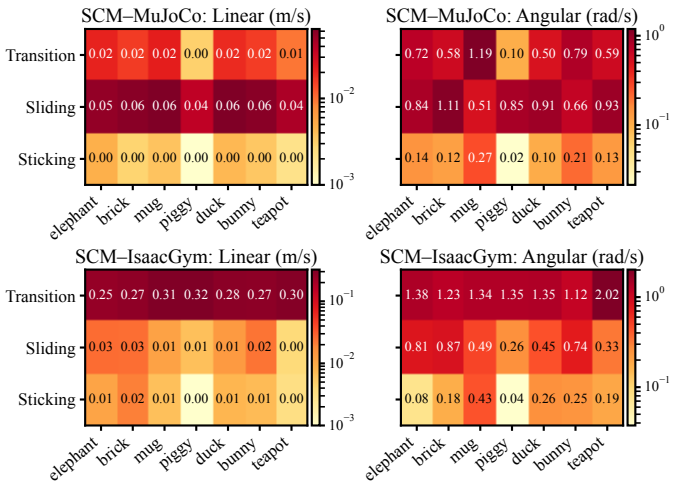


Fig. 11. Mean SCM prediction errors by object and contact state (log scale).

errors below $0.3\,\mathrm{rad/s}$ in sticking and about $1\,\mathrm{rad/s}$ otherwise; outside transition they vary little with n_c. Per object (Fig. 11), the SCM–MuJoCo linear errors stay below $0.07\,\mathrm{m/s}$ for all objects and only the IsaacGym transition row is elevated. For SCM, we set $\epsilon = 10^{-8}$ and $T = 10^{-4}\,\mathrm{N\,s}$ and report the one-step prediction of (27) with $K = 3$ sweeps at $h = 0.002\,\mathrm{s}$. Initial poses are obtained by native MuJoCo settling.

We then evaluate whether the ideal contact trajectories generated by each model can achieve the manipulation goals under the success criteria in (44). The results are shown in Table V. M1 and M2 denote the native contact models of MuJoCo (QP-based) and IsaacGym (PhysX), each used as the CSO model and rolled out in its own engine. Ideal control applies the CSO wrench $\boldsymbol{\lambda}^*$ directly at the optimal contact for one step h before replanning, bypassing the robot; we report manipulation success, planning time, and execution steps under synchronous inference without observation latency. Each model is evaluated on 100 rotation and 100 flipping trials with the setup and success criteria of Section V-A. Planning time and execution steps are mean $\pm$ std, with planning time pooled over planning rounds and execution steps taken over successful trials. All models reach an ideal success of 0.99–1.00, so each yields contact guidance that can succeed despite the errors in Table IV; inside CSO, however, SCM plans an order of magnitude faster and attains the higher deployed success in both engines.

3) Counterfactual Analysis of the Ranking Strategy: A1–A2 evaluate alternative contact-selection policies at the task level. We further assess the ranking strategy at individual decision points using matched counterfactual rollouts.

Inspired by reinforcement learning, we define a value for each contact-selection decision made by the ranking strategy. Let $r_T = \mathbb{I}(\text{success})$ be the terminal reward. For a state s_t and ranking decision b, the discounted rollout value is

$$J(s_t, b) = \mathbb{E}_{\tau\sim\pi(\cdot|s_t,b)}\left[\eta_{\text{cf}}^{T}\mathbb{I}(\text{success})\right], \qquad \eta_{\text{cf}} = 0.99, \tag{45}$$

where T counts the actual execution steps from s_t to trial termination, including all detour and contact-transition steps, rather than contact-selection events, and the discount factor is denoted by η_{cf} to avoid confusion with the ranking phase variable γ_t in (36).

We sample 200 SCSP rollout trajectories and, in each, 10 decision points where the optimal contact location p_r^* differs from p_{near} (2000 decision points in total), and record the ranking strategy's decisions. At each selected state, we reverse the recorded decision, force the robot to execute the alternative contact strategy, and then resume SCSP until termination to obtain the reward and estimate the corresponding value. For example, during a contact-free phase, if SCSP selects the nearby contact, we instead force the robot to detour to p_r^*. During a contact phase, if SCSP decides to break the current contact and approach p_r^*, we instead force it to maintain the current contact. Let J_{RS} and J_{cf} denote the rollout values for the original and reversed decisions at s_t, respectively. Because SCSP replans at high frequency, a single-step decision deviation may have only a small effect on the overall trajectory. We therefore protect the perturbed decision from ranking overrides until the selected contact is established and evaluate its effect using the normalized advantage, defined as

$$\widetilde{A}(s_t) = \begin{cases} \dfrac{J_{\text{RS}} - J_{\text{cf}}}{\max\{J_{\text{RS}}, J_{\text{cf}}\}}, & \max\{J_{\text{RS}}, J_{\text{cf}}\} > 0, \\ 0, & \text{otherwise.} \end{cases} \tag{46}$$

Here $\widetilde{A} \in [-1, 1]$, and $\widetilde{A} > 0$ indicates that the ranking strategy is preferred. A value of 1 (-1) means that only the RS (counterfactual) rollout succeeds. For two successful rollouts, this normalization depends only on the difference in their remaining completion steps, removing the common discount scale associated with long remaining trajectories.

We report the physical contact-switching frequency f_{switch}, CSO-optimal contact-point switching frequency f_r^*, and selection probabilities $P(p_{\text{near}})$ and $P(p_r^*)$. These statistics are computed over the original sampled trajectories. Table VI shows relatively balanced selection probabilities $P(p_{\text{near}})$ and $P(p_r^*)$ across the sampled trajectories, indicating that both contact strategies are extensively used. The CSO-optimal contact p_r^* changes at approximately $5\,\mathrm{Hz}$, while physical contact switches occur at only about $0.2\,\mathrm{Hz}$. This suggests that the ranking strategy frequently mediates contact selection and filters changes in contact guidance. The fraction of positive advantages and the mean normalized advantage are particularly high for flipping (90.68% and 0.36, respectively). These results are consistent with our observations: for rotation, the ranking strategy primarily improves efficiency by reducing detours; for flipping, it also helps maintain contact stability by avoiding frequent contact breaks.

C. Real-world Experiments

Finally, we validate the robustness of SCSP in real-world experiments. Most model-based methods perform well in simulation, but in real-world experiments, they often rely on highly accurate perception systems, such as motion capture. The main challenge in replacing exact perception with purely vision-based perception is that errors in pose observation can perturb contact estimation, leading to incorrect inference of contact states and consequently inaccurate contact dynamics.

TABLE IV
SCM VELOCITY ERRORS RELATIVE TO MUJOCO AND ISAACGYM (MEAN ± STD; LINEAR IN m/s AND ANGULAR IN rad/s).

Comparison	Metric	Contact transition			Sliding			Sticking			Rollout	
		$n_c = 1$	$n_c = 2$	$n_c \geq 3$	$n_c = 1$	$n_c = 2$	$n_c \geq 3$	$n_c = 1$	$n_c = 2$	$n_c \geq 3$	Rotation	Flipping
SCM–MuJoCo	Linear	0.01 ± 0.01	0.03 ± 0.02	0.02 ± 0.02	0.04 ± 0.02	0.06 ± 0.03	0.06 ± 0.03	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.01 ± 0.03	0.02 ± 0.14
SCM–MuJoCo	Angular	0.46 ± 0.56	0.67 ± 0.57	1.09 ± 1.02	0.89 ± 0.65	0.91 ± 0.41	0.78 ± 0.47	0.08 ± 0.06	0.10 ± 0.01	0.19 ± 0.06	0.36 ± 0.87	0.57 ± 2.58
SCM–IsaacGym	Linear	0.42 ± 0.35	0.37 ± 0.32	0.37 ± 0.33	0.01 ± 0.00	0.01 ± 0.01	0.02 ± 0.01	0.00 ± 0.00	0.01 ± 0.00	0.01 ± 0.01	0.04 ± 0.12	0.14 ± 0.75
SCM–IsaacGym	Angular	2.25 ± 2.16	1.80 ± 1.76	1.69 ± 1.47	0.29 ± 0.14	0.45 ± 0.17	0.73 ± 0.31	0.12 ± 0.08	0.26 ± 0.01	0.23 ± 0.13	4.00 ± 17.08	9.73 ± 37.64

TABLE V
CSO CONTACT-GUIDANCE PERFORMANCE.

Metric	MuJoCo		IsaacGym	
	M1	SCM	M2	SCM
Success (ideal)	1.00	1.00	0.99	1.00
Planning time (ms)	27.76 ± 5.68	2.18 ± 0.75	22.82 ± 3.00	2.30 ± 0.79
Execution steps	160.74 ± 98.06	216.96 ± 126.33	305.09 ± 287.05	379.00 ± 465.02
Success (manip.)	0.46	0.85	0.76	0.88

TABLE VI
RANKING-STRATEGY COUNTERFACTUAL ANALYSIS.

Task	$P(p_{\text{near}})$	$P(p_r^*)$	f_{switch} (Hz)	f_r^* (Hz)	$P(\widetilde{A} > 0)$ (%)	$\widetilde{A}$
Rotation	0.353	0.647	0.203	5.51	71.42	0.17 ± 0.28
Flipping	0.450	0.550	0.195	4.40	90.68	0.36 ± 0.42

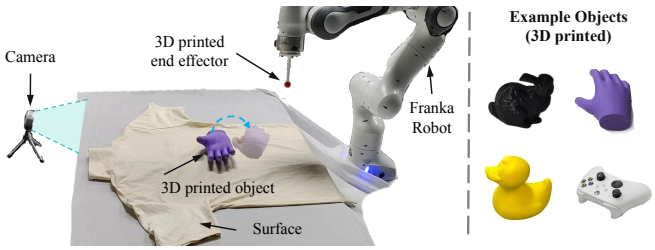


Fig. 12. Hardware setup of the real-world experiments.

To the best of our knowledge, our method is the first model-based approach that can achieve purely vision-based, complex geometries, model mismatch contact-rich manipulation.

1) Experimental Setup: We conduct experiments using a Franka Emika Panda equipped with a 3D-printed end-effector, and arbitrarily select 4 irregularly shaped objects: Duck, Bunny, Hand, and Xbox Controller, as shown in Fig. 12. We evaluate on-ground rotation on Duck and Bunny, and on-ground flipping on Bunny, Hand, and Xbox Controller. The object CAD models are obtained from the ContactDB dataset and online repositories. To validate the robustness of our method under inaccurate dynamics, we provide only rough estimates of the object's physical parameters such as mass and friction coefficients. Moreover, instead of relying on precise motion capture for exact perception, we perform state estimation by `FoundationPose` [46] using a single-view camera (Realsense D435i). In real-world experiments, precise object–environment contact locations are not directly observable. To support visualization and coarse kinematic alignment, we maintain a MuJoCo proxy of the table and end-effector, as shown in Fig. 13. We calibrate only the object

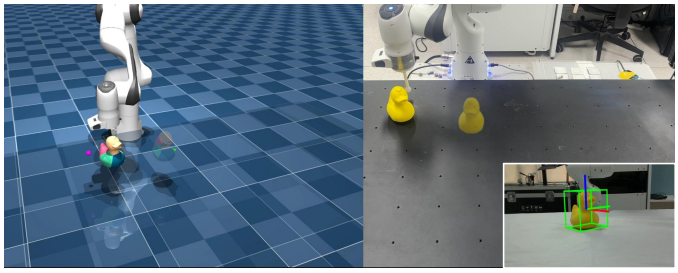


Fig. 13. Perception method in the real-world experimental setting. Object pose is detected using `FoundationPose` and filtered via a Kalman filter. Contact estimation is performed using a `MuJoCo` implementation that approximates the experimental environment.

scale and positional offsets $[\Delta x, \Delta y, \Delta z]$ from kinesthetic teaching. The proxy is not used as a physical object-contact model: the rendered object pose is updated in real time from `FoundationPose`, and no object collision geometry is defined in the simulator. The goal pose range, success conditions, and maximum execution steps for the task are identical to those in the simulation experiments. Sampling and DyWA use the same settings as in the previous section, and the outputs of DyWA are executed using an impedance controller.

The OSC controller is implemented in `C++`. Unlike in simulation experiments, control in real-world deployment is subject to disturbances, leading to control lag and steady-state errors. To address this, we designed an adaptive controller for precise control, which we refer to as the Incremental OSC. We introduce an intermediate position vector $\boldsymbol{p}_{\text{inc}} = \boldsymbol{p}_{\text{inc}} + \boldsymbol{K}_{\text{inc}}\boldsymbol{u}$, and replace $\boldsymbol{e}_{\text{pos}}$ with $\boldsymbol{e}_{\text{pos}} = \boldsymbol{p}_{\text{inc}} - \boldsymbol{p}_{\text{ee}}$. The steady-state error is adaptively eliminated under the high-frequency self-integrating effect. In the experiment, we set $\boldsymbol{K}_{\text{inc}} = 0.01\boldsymbol{I}_3$, $\boldsymbol{K}_x = \text{diag}(500, 500, 500, 10, 10, 10)$ and $\boldsymbol{D}_x = \sqrt{2\boldsymbol{K}_x}$.

2) Results and Discussion: We first evaluate the success rate of SCSP across different objects and tabletop materials to assess its robustness under model inaccuracies. The experimental results are shown in Fig. 14, 15, and Table VII. They indicate that our method maintains strong performance under perception noise and model mismatch. In Table VII, the 95% CI rows report two-sided Wilson confidence intervals for SR. Bold denotes the highest SR or the lowest FR, TR, and mean ET under the same condition.

We conducted each experiment on two different tabletop materials: smooth and rough as in Fig. 16. For the on-ground rotation experiments, the smooth surface denotes a tabletop with low friction, whereas the rough surface denotes cotton clothes. For the on-ground flipping experiments, which require

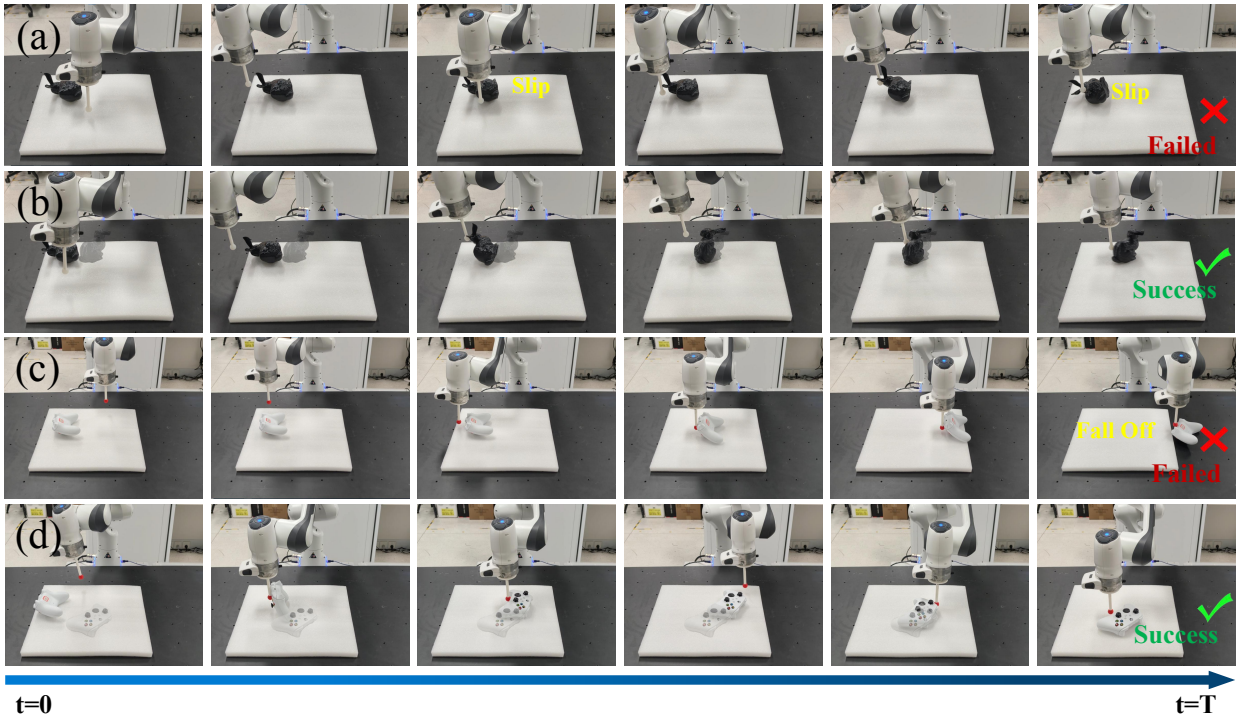


Fig. 14. Snapshots of nonprehensile manipulation in real-world experiments. (a) **DyWA** failed to flip the bunny. (b) **SCSP** succeeds in flipping the bunny. (c) **Sampling** failed by pushing the Xbox controller to fall off the foam padding. (d) **SCSP** succeeds in flipping the Xbox controller.

TABLE VII
REAL-WORLD MANIPULATION PERFORMANCE ON SMOOTH AND ROUGH SURFACES (30 TRIALS PER CONDITION).

Surface	Metric	Rotation						Flipping								
		Duck			Bunny			Bunny			Hand			Xbox		
		Sampling	DyWA	**SCSP***	Sampling	DyWA	**SCSP***	Sampling	DyWA	**SCSP***	Sampling	DyWA	**SCSP***	Sampling	DyWA	**SCSP***
Smooth	SR (%)	60	50	**100**	40	40	**100**	0	0	**46.7**	13.3	0	**70**	10	13.3	**80**
	FR (%)	6.7	10	**0**	6.7	10	**0**	20	**0**	**0**	23.3	6.7	**0**	16.7	6.7	**0**
	TR (%)	33.3	40	**0**	53.3	50	**0**	80	100	**53.3**	63.4	93.3	**30**	73.3	80	**20**
	95% CI	[42.3, 75.4]	[33.2, 66.8]	[88.6, 100.0]	[24.6, 57.7]	[24.6, 57.7]	[88.6, 100.0]	[0.0, 11.4]	[0.0, 11.4]	[30.2, 63.9]	[5.3, 29.7]	[0.0, 11.4]	[52.1, 83.3]	[3.5, 25.6]	[5.3, 29.7]	[62.7, 90.5]
	ET (s)	15.31 ± 10.17	**15.19 ± 10.31**	18.21 ± 6.42	17.12 ± 8.83	17.21 ± 8.22	**14.57 ± 5.69**	–	–	**22.34 ± 12.41**	25.90 ± 10.87	–	**20.19 ± 11.59**	19.86 ± 2.15	27.31 ± 9.67	**17.89 ± 10.09**
Rough	SR (%)	53.3	46.7	**96.7**	40	36.7	**100**	0	0	**66.7**	10	10	**83.3**	16.7	20	**83.3**
	FR (%)	6.7	13.3	**0**	3.3	6.7	**0**	10	20	**0**	20	10	**0**	16.7	16.7	**0**
	TR (%)	40	40	**3.3**	56.7	56.6	**0**	90	80	**33.3**	70	80	**16.7**	66.6	63.3	**16.7**
	95% CI	[36.1, 69.8]	[30.2, 63.9]	[83.3, 99.4]	[24.6, 57.7]	[21.9, 54.5]	[88.6, 100.0]	[0.0, 11.4]	[0.0, 11.4]	[48.8, 80.8]	[3.5, 25.6]	[3.5, 25.6]	[66.4, 92.7]	[7.3, 33.6]	[9.5, 37.3]	[66.4, 92.7]
	ET (s)	18.08 ± 8.01	**16.88 ± 8.90**	19.25 ± 9.58	18.41 ± 6.73	25.25 ± 10.05	**15.56 ± 7.81**	–	–	**21.21 ± 9.97**	26.61 ± 11.32	27.99 ± 9.73	**20.30 ± 5.83**	18.90 ± 13.26	21.08 ± 11.35	**18.39 ± 11.86**

SR: success rate; FR: fall-out rate; TR: timeout rate; ET: execution time.

TABLE VIII
FAILURE CASE ANALYSIS IN REAL-WORLD EXPERIMENTS.

Metric	Trials					Recovery Rate				
	Sampling	DyWA	**SCSP**	A8	A9	Sampling	DyWA	**SCSP**	A8	A9
Perception	2/300	8/300	1/300	1/300	0/300	0/2	0/8	0/1	0/1	–
Contact Estimation	300/300	300/300	300/300	300/300	300/300	73/300	65/300	248/300	58/300	37/300
Slippage	110/300	147/300	108/300	158/300	142/300	20/110	20/147	98/108	26/158	19/142
Control Saturation	8/300	45/300	27/300	85/300	11/300	2/8	7/45	25/27	32/85	10/11
Poor Contact Selection	62/300	15/300	15/300	90/300	14/300	12/62	3/15	3/15	0/90	0/14

greater friction, we use foam padding as the rough surface and cotton cloth as the smooth surface. Note that our method does not measure friction coefficients, object mass, or other physical parameters, but instead uses coarse estimates. Additionally, biases in contact estimation and noise in object pose estimation can interfere with the decision-making of SCSP. The experiment results suggest that SCSP can effectively cope with model inaccuracies and disturbances, which is typically challenging for model-based methods.

As shown in Table VII, SCSP achieved a 99.2% success rate (119/120) in the on-ground rotation experiments with complex object geometries and also showed strong performance in the flipping tasks, whereas the comparison methods failed in nearly all flipping experiments. The execution times indicate that SCSP also completes the tasks relatively efficiently, with mean times of approximately 15–30 s across the tested conditions. SCSP has the highest estimated success rate under every condition, and its 95% Wilson interval does not overlap with that of either comparison method in any condition. We report these intervals as uncertainty estimates for each success rate rather than as a formal hypothesis test between methods. We also classify all unsuccessful trials by their terminal outcomes: timeout or falling off the table. Most unsuccessful trials terminate due to timeout.

We further examine five factors associated with task failures: perception failure, contact estimation error, slippage, control saturation, and poor contact selection. Perception failure denotes target tracking loss in `FoundationPose` with an unrecoverable pose estimate. It is rare and occurs only under severe occlusion when the robot reaches to retrieve an object near the table edge. Contact estimation error includes

missed physical contact and reported penetration before actual contact. Slippage denotes unstable robot–object contact with relative sliding. It particularly affects flipping, where complex object geometry and smooth 3D-printed surfaces can prevent stable fingertip contact and compromise the planned motion. Surface friction also influences flipping performance, with SCSP achieving higher success rates on the rough surface. Control saturation occurs when adaptive-controller overshoot produces excessive contact commands that reach the robot's control limits and may topple the object. Poor contact selection is defined as selecting the same contact location again after three consecutive unsuccessful manipulation attempts at that location. We record the occurrence frequencies and task success rates among affected trials for SCSP, DyWA, Sampling, A8, and A9. Table VIII summarizes the occurrence frequencies and recovery rates. DyWA failures mainly involve contact slippage and violations of the Franka acceleration limits during pressing. Sampling failures involve unstable contact locations.

Contact estimation errors can falsely indicate penetration or miss existing physical contact. SCM sets $\phi \equiv 0$ to suppress spurious repulsive wrenches induced by estimated penetration, while CPO attracts the end-effector toward an inward-offset target rather than the estimated surface, helping avoid local minima above the physical surface. We compare A8, which removes the $\phi \equiv 0$ condition, and A9, which replaces the target in (38) with the surface point $\boldsymbol{p}_{\text{ref}}$. Table VIII summarizes the comparison. A9 sometimes stalls above the object without establishing contact, leading to timeout. A8 is particularly sensitive to contact estimation errors: when `FoundationPose` estimates an object pose penetrating the ground, we observe abrupt changes in A8's predicted object positions, sometimes causing premature contact loss during flipping and subsequent object toppling.

We further conduct continuous manipulation experiments, as shown in Fig. 15. During manipulation, we repeatedly perturb the object pose to evaluate the real-time performance, closed-loop planning capability, and robustness of SCSP under accumulated perception errors. Under repeated perturbations, both the pose estimated from `FoundationPose` and the contact estimation become biased. However, SCSP exhibits strong replanning capability and consistently completes the continuous manipulation tasks under disturbances.

We combine the preceding experimental results to analyze the source of SCSP's robustness to model mismatch. We conclude that: 1) the ablation study reveals that optimality of the contact prior is not necessary. Although the prior is important for contact planning, even a coarse contact location selection as a prior can still bring substantial performance improvement. This observation also provides insight into the role of the contact-selection model: how to properly connect the prior with the planning framework seems to be more important than the optimality and accuracy of the prior itself. The designs of CSO and the ranking strategy are both developed with this perspective. 2) CPO has a strong closed-loop nature. The real-time performance and robustness of CIMPC methods have been thoroughly validated in previous work. CPO is a prior-guided contact planning framework built upon the CIMPC formulation. It inherits the robustness of CIMPC while improving manipulation diversity. Therefore, the SCSP framework is able to achieve robust performance.

TABLE IX
COMPUTATION TIME OF SCSP

Metric	Value
Offline sampling cost time (s)	4.82 ± 3.85
Exchange cost time (ms)	0.002 ± 0.01
CSO solving time (ms)	2.35 ± 0.73
RS cost time (s)	0.01 ± 0.00
Average CPO solving time (ms)	8.68 ± 0.31

VI. GENERALIZATION TO MULTIPLE TASKS

We further implement SCSP in different scenarios to evaluate its cross-task generalization. Using multi-contact and articulated-object manipulation tasks as examples, we demonstrate online contact selection for underactuated objects.

A. Case Study I: Multi-Contact Task

SCSP provides a potential framework for extending contact selection and planning to multi-contact tasks, rather than a general solution. We illustrate the generalization of SCSP with following example:

1) Soccer Dribbling: We use a soccer dribbling task to demonstrate the performance of SCSP in dynamic scenarios. In this task, SCSP plans contact with the ball using one of the two feet and drives it toward the goal through multi-contact manipulation. The main challenges lie in the highly dynamic behavior of the ball, which makes stable and sustained contact difficult, and in coordinating the two feet effectively. We read the ball position $\boldsymbol{p}_k$ and estimate its velocity $\boldsymbol{v}_k$ from the simulation using a Kalman filter. The 3D model of the ball is then input to the SCSP to compute the actions $\boldsymbol{u} \in \mathbb{R}^{2\times 4}$ for both feet, where the control vector for each foot is $[x, y, z, \theta_{\text{yaw}}]$. We define activation triggers separately for the left and right feet: using the midpoint between both feet as the center, the space is divided into left and right half-spaces. When $\boldsymbol{x}_k$ appears in a half-space, the corresponding left or right foot is activated. Additionally, workspace constraints (represented as semi-transparent cubes) are imposed on each foot to prevent singular gait patterns. The number of CSO sampling points is set to $n_s = 70$, and points close to the ground $\{p_i \in \mathcal{I}_{\text{obj}} \mid z_i < 0.02\text{m}\}$ are masked out. We randomly selected 10 goal positions for the soccer ball, and SCSP continuously controlled the robot's feet to dribble the ball to each target, achieving a 100% success rate. The experimental results are shown in Fig. 17. This experiment serves as a toy example, simulating the approaching and contact phases in humanoid soccer dribbling. In the future, we plan to extend SCSP to more complex motion planners to realize a more complete demonstration [47], [48].

2) Tilted Pushing: SCSP can be extended to manipulation tasks in uneven environments, such as tilted pushing. Stable object poses on a slope are inherently fragile, requiring accurate contact location selection and online replanning. We compare open-loop methods, which plan a contact sequence

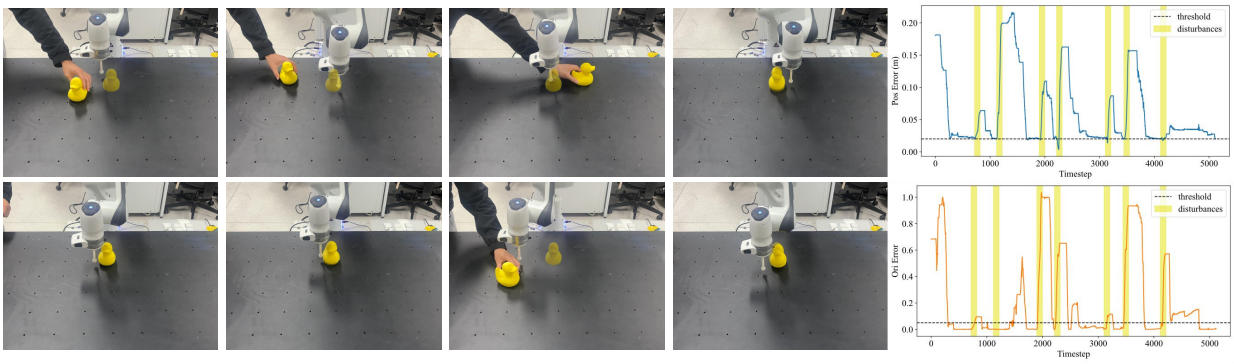


Fig. 15. Snapshots of manipulation under external disturbances. The object's pose is continuously perturbed during manipulation to demonstrate SCSP's real-time robustness under such disturbances. The panels on the right show the temporal evolution of position and orientation errors.

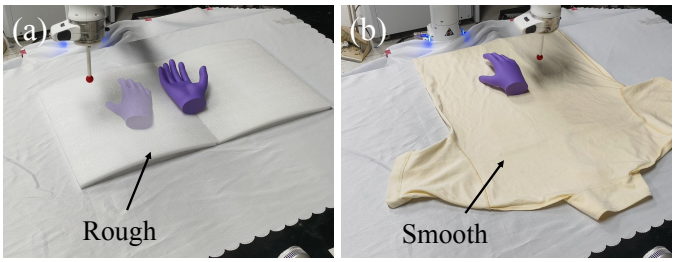


Fig. 16. Robustness validation of SCSP on surfaces with different friction. (a) Foam. (b) Cotton clothes.

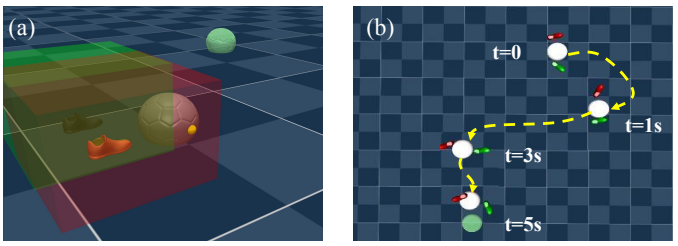


Fig. 17. Soccer dribbling task. (a) Experiment setup. The task is to kick the soccer to the target (translucent). Red and green cubes show foot workspaces, and the yellow point indicates CSO-planned locations. (b) Dribbling trajectory.

offline, with SCSP, which replans online during execution. As the baseline, we use STOCS-3D to compute an offline sequence of contact locations and contact forces, and execute this sequence using our CPO. Experiments are conducted in `IsaacGym` with a 15° slope and a friction coefficient of 0.1. For SCSP, we set $\hat{\lambda}^n_{\max} = 0.5$, while keeping all other parameters the same as in Section V-A. We selected the teapot and duck (used in previous experiments) and randomly set 10 stable poses on the slope as goal poses, and pushed and rotated the objects from below to reach these goals. The results in Table X and Fig. 18 show that the real-time replanning capability of SCSP provides substantially better robustness.

3) Grasping & Dexterous Manipulation: SCSP can also be applied to predict grasping poses, such as bimanual grasping and multi-finger grasping. Following [49], we use the Grasp Wrench Boundary (GWB) [50] to select grasping regions and identify the top-K pairs of potential valid regions for grasping. Subsequently, sampling of the candidate set $\mathcal{I}_{\text{obj}}$ is performed within these regions. The objective of the CSO inner loop

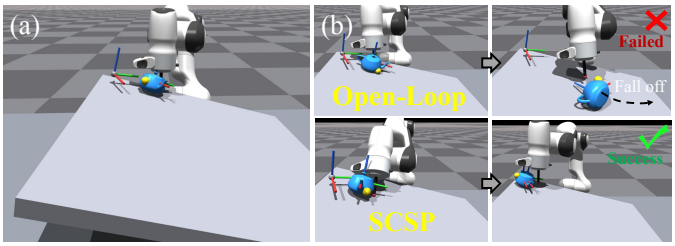


Fig. 18. Experiment setup. The task is to push an object on a surface tilted at 15°. (a) The open-loop method fails when execution errors occur, causing the teapot to fall off the tilted surface. (b) SCSP can replan online and thus successfully complete the dynamic task.

optimization is then transformed to force closure:

$$\begin{aligned} \min_{\hat{\boldsymbol{\lambda}}_r \in \mathcal{K}} \quad & \sum_{j=1}^{n_d} \left\| \xi \boldsymbol{w}^{(j)} - \boldsymbol{J}_o \hat{\boldsymbol{\lambda}}_r^{(j)} \right\|_2^2 + w_\lambda \sum_{j=1}^{n_d} \left\| \hat{\boldsymbol{\lambda}}_r^{(j)} \right\|_2^2 \\ \text{s.t.} \quad & \sum_{i=1}^{n_c} \hat{\lambda}_{r,i}^{(j),n} \geq \zeta, \ \forall j = 1, \ldots, n_d. \end{aligned} \tag{47}$$

where n_c is the number of robot contact points, n_d denotes the number of sampled disturbance wrenches, $\boldsymbol{w}^{(j)} \in \mathbb{R}^6$ represents the j-th disturbance wrench, and $\boldsymbol{J}_o$ is the stacked object contact Jacobian. $\hat{\boldsymbol{\lambda}}_r^{(j)}$ denotes the robot contact force corresponding to disturbance $\boldsymbol{w}^{(j)}$, with normal component $\hat{\lambda}_{r,i}^{(j),n}$ at contact i. ξ is the disturbance scaling factor, ζ the minimum total normal force required to maintain an active grasp and $w_\lambda = 0.1$. During each rollout, the CSO in SCSP solves (47) for the nearest points $\boldsymbol{p}^i_{\text{near}}$ to the robot end-effector or Allegro fingers additional to the top 5 pairs of potential grasping pairs, thereby searching for grasping poses online. We use an open-loop baseline that computes the optimal grasping locations offline using CSO and executes them online with CPO for comparison. For bi-manual and multi-finger grasping, we selected five objects each, placed them on a pedestal (black), randomly perturbed their yaw angles θ_{yaw} and scaled their sizes by a random factor $\varsigma \in \{0.5, 1, 1.5, 2\}$, conducting 20 trials for each task. The results are shown in Table X and Fig. 19. SCSP substantially outperforms the open-loop baseline in bi-manual grasping and achieves a slight advantage in multi-finger grasping. In the additional successful cases, we observed small in-hand adjustments after grasp slippage.

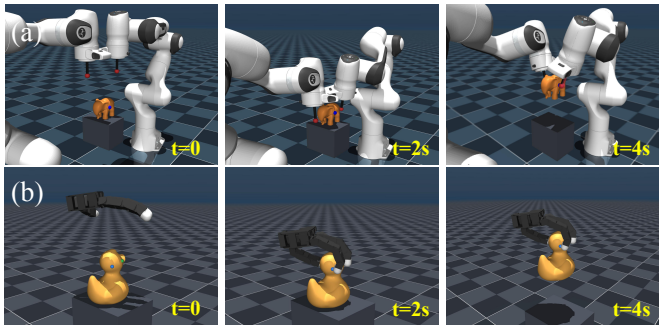


Fig. 19. Snapshots of the multi-contact grasping tasks. (a) Bi-manual grasping of an elephant with the size scaling factor $\varsigma = 1$. (b) Multi-finger grasping of a duck with $\varsigma = 1.5$.

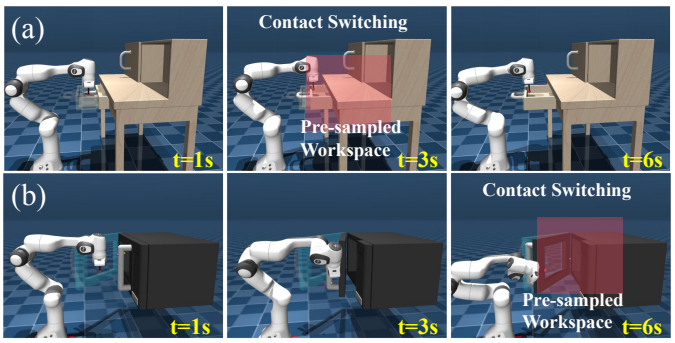


Fig. 20. Snapshots of articulated-object manipulation. Given only the target pose, SCSP reasons online, selects contact locations, and switches contacts when the current contact leaves the predefined workspace. (a) Opening a drawer. (b) Opening the microwave door.

B. *Case Study II: Articulated Object Manipulation*

SCSP uses active contact selection to perform articulated object manipulation. We evaluate this capability on a drawer and a microwave door, which require sequential contact interactions. The tasks are to pull the drawer to a target displacement and to open the microwave door to a target angle. In both tasks, the object is semantically segmented offline into different parts, such as handles, while non-manipulable parts are excluded. From the current end-effector pose, we coarsely sample a 3D Cartesian region. Contact points outside it may require a large change in contact posture and are masked. The drawer interior and the microwave-door interior are also masked before the drawer or door is pulled open. For the drawer, we directly compute the final pulled-out pose as the goal pose. For the microwave, which involves rotational motion, we set the goal pose of each rollout as the `SLERP` interpolation between the current state and the desired door state. The parameter and cost settings of SCSP are the same as those in Section V-A. The solver for CPO is an MPPI-based motion planner [51]. As a baseline, we use an open-loop method that plans the contact sequence offline using CSO and executes it online with CPO, in order to demonstrate the effectiveness of online contact selection. We randomly perturbed the distance and yaw angle between the drawer or microwave and the Franka base by $\Delta d \in [-0.1, 0.1]\,\mathrm{m}$ and $\Delta\theta_{\text{yaw}} \in \left[-\frac{\pi}{4}, \frac{\pi}{4}\right]$ and conducted 10 trials for each object. The experimental results are shown in Fig. 20, Table X and the website videos. When the contact point on the handle leaves this region, the contact switches to the drawer panel or to the inner wall of the microwave door. SCSP succeeds in 20/20 trials versus 15/20 for the open-loop baseline. Offline sequences fail under slippage or near-singular configurations. Because CSO is object-centric, its selected contact may violate robot kinematic limits; the ranking strategy substitutes a nearby feasible candidate and enables recovery.

VII. Discussion & Limitations

The theoretical and experimental results support the usefulness of explicit contact selection, while also exposing several limitations of the current implementation. CSO provides contact-location priors through mesh discretization and a lightweight surrogate contact model. This design is computationally efficient, but its accuracy decreases in complex contact configurations because it assumes a fixed object–environment contact set, restores inter-contact coupling only through three unrolled Gauss–Seidel sweeps, and represents contacts as points. CSO therefore does not provide sequential, high-fidelity state prediction and is not directly suited to complex end-effectors or deformable objects. Higher-fidelity contact models such as [52] are a potential direction for these capabilities, although their interfaces and task domains differ from SCM and they are not drop-in replacements. General multi-contact manipulation remains limited by scalability: selecting n_{r} robot contacts from n_s candidates grows combinatorially, e.g., $\binom{n_s}{n_{\text{r}}}$ in the unordered case, and tasks with strong geometric coupling among contacts, such as power grasping, are beyond the demonstrated precision-grasping case. Reducing the CSO search space, as done with the GWB-based regions in Section VI, is a first step toward this direction and is left for future work.

CPO generates trajectories for contact-free and contact-rich stages using model-predictive control. Its implementation may be less suitable for tasks such as loco-manipulation and may not maintain continuous surface contact during contact switching. Future work can replace the rule-based ranking strategy with a learned representation and can combine CPO with a more expressive motion planner such as [53].

The experiments also suggest that the interface between contact selection and trajectory optimization is important: CPO can use a nearby contact point when the lowest-cost sampled point is difficult to execute. These results do not establish surrogate accuracy beyond the evaluated tasks; robustness claims therefore apply only to tested settings.

VIII. Conclusion

This work presented SCSP, a cascaded framework that separates contact-location selection from contact-trajectory optimization for contact-rich manipulation. CSO evaluates a finite set of sampled surface points with a surrogate contact model, and CPO uses the resulting contact prior together with a ranking rule to generate approaching, contact, and contact-switching trajectories. Simulations and hardware experiments show that this decomposition can improve manipulation diversity and task success under the tested geometric and modeling

TABLE X
SUCCESS TRIALS ACROSS DIFFERENT TASKS.

Metric	Soccer Dribbling		Tilted Pushing		Bi-Manual Grasping		Multi-finger Grasping		Articulated Object Manipulation	
	Open-Loop	SCSP	Open-Loop	SCSP	Open-Loop	SCSP	Open-Loop	SCSP	Open-Loop	SCSP
Success Trials	/	**10/10**	0/20	**18/20**	0/20	**15/20**	15/20	**16/20**	15/20	**20/20**

conditions. The results also indicate that SCSP remains limited by the point-contact and fixed-contact-set assumptions of the surrogate model, as well as by the need for a one-time MuJoCo calibration in the vision-based hardware experiments.

REFERENCES

[1] C. Zito *et al.*, "Two-level RRT planning for robotic push manipulation," in *Proc. IROS*, 2012, pp. 678–685.
[2] A. Wu *et al.*, "R3T: Rapidly-exploring random reachable set tree for optimal kinodynamic planning of nonlinear hybrid systems," in *Proc. ICRA*, 2020, pp. 4245–4251.
[3] J. A. Haustein *et al.*, "Kinodynamic randomized rearrangement planning via dynamic transitions between statically stable states," in *Proc. ICRA*, 2015, pp. 3075–3082.
[4] C. Chen *et al.*, "Trajectotree: Trajectory optimization meets tree search for planning multi-contact dexterous manipulation," in *Proc. IROS*, 2021, pp. 8262–8268.
[5] X. Cheng *et al.*, "Contact mode guided motion planning for quasidynamic dexterous manipulation in 3D," in *Proc. ICRA*, 2022, pp. 2730–2736.
[6] B. Aceituno-Cabezas *et al.*, "A global quasi-dynamic model for contact-trajectory optimization in manipulation," 2020.
[7] T. Marcucci *et al.*, "Approximate hybrid model predictive control for multi-contact push recovery in complex environments," in *Proc. Humanoids*, 2017, pp. 31–38.
[8] F. R. Hogan *et al.*, "Feedback control of the pusher-slider system: A story of hybrid and underactuated contact dynamics," in *Proc. WAFR*, 2020, pp. 800–815.
[9] M. Kim *et al.*, "Pre- and post-contact policy decomposition for non-prehensile manipulation with zero-shot sim-to-real transfer," in *Proc. IROS*, 2023, pp. 10 644–10 651.
[10] J. Liang *et al.*, "Learning preconditions of hybrid force-velocity controllers for contact-rich manipulation," in *Proc. CoRL*.
[11] W. Zhou *et al.*, "Learning to grasp the ungraspable with emergent extrinsic dexterity," in *Proc. CoRL*, 2023, pp. 150–160.
[12] X. Zhang *et al.*, "Learning generalizable pivoting skills," in *Proc. ICRA*, 2023, pp. 5865–5871.
[13] W. Zhou *et al.*, "HACMan: Learning hybrid actor-critic maps for 6D non-prehensile manipulation," in *Proc. CoRL*, 2023, pp. 241–265.
[14] Y. Cho *et al.*, "Corn: Contact-based object representation for nonprehensile manipulation of general unseen objects," *arXiv:2403.10760*, 2024.
[15] B. Jiang *et al.*, "Hacman++: Spatially-grounded motion primitives for manipulation," *arXiv:2407.08585*, 2024.
[16] H. Le *et al.*, "Enhancing exploration with diffusion policies in hybrid off-policy RL: Application to non-prehensile manipulation," *IEEE Robot. Autom. Lett.*, 2025.
[17] J. Lyu *et al.*, "Dywa: Dynamics-adaptive world action model for generalizable non-prehensile manipulation," in *Proc. ICCV*, 2025, pp. 11 058–11 068.
[18] A. Aydinoglu *et al.*, "Consensus complementarity control for multi-contact MPC," *IEEE Trans. Robot.*, 2024.
[19] Y. Li *et al.*, "On the surprising robustness of sequential convex optimization for contact-implicit motion planning," *arXiv:2502.01055*, 2025.
[20] V. Kurtz *et al.*, "Inverse dynamics trajectory optimization for contact-implicit model predictive control," *Int. J. Robot. Res.*, vol. 45, no. 1, pp. 23–40, 2026.
[21] T. Pang *et al.*, "Global planning for contact-rich manipulation via local smoothing of quasi-dynamic contact models," *IEEE Trans. Robot.*, vol. 39, no. 6, pp. 4691–4711, 2023.
[22] W. Jin, "Complementarity-free multi-contact modeling and optimization for dexterous manipulation," *arXiv:2408.07855*, 2024.
[23] M. Zhang *et al.*, "Simultaneous trajectory optimization and contact selection for contact-rich manipulation with high-fidelity geometry," *IEEE Trans. Robot.*, 2025.
[24] W. Yang *et al.*, "Contactsdf: Signed distance functions as multi-contact models for dexterous manipulation," *arXiv:2408.09612*, 2024.
[25] S. Venkatesh *et al.*, "Approximating global contact-implicit MPC via sampling and local complementarity," *arXiv:2505.13350*, 2025.
[26] M. Posa *et al.*, "A direct method for trajectory optimization of rigid bodies through contact," *Int. J. Robot. Res.*, vol. 33, pp. 69–81, 2014.
[27] P. M. Wensing *et al.*, "Optimization-based control for dynamic legged robots," *IEEE Trans. Robot.*, vol. 40, pp. 43–63, 2023.
[28] D. Stewart *et al.*, "An implicit time-stepping scheme for rigid body dynamics with Coulomb friction," in *Proc. ICRA*, 2000, pp. 162–169.
[29] M. Anitescu, "Optimization-based simulation of nonsmooth rigid multi-body dynamics," *Math. Program.*, vol. 105, no. 1, pp. 113–143, 2006.
[30] H. Bui *et al.*, "Push anything: Single- and multi-object pushing from first sight with contact-implicit MPC," *arXiv:2510.19974*, 2025.
[31] H. Suh *et al.*, "Dexterous contact-rich manipulation via the contact trust region," *arXiv:2505.02291*, 2025.
[32] Y. Jiang *et al.*, "Robust model-based in-hand manipulation with integrated real-time motion-contact planning and tracking," *arXiv:2505.04978*, 2025.
[33] Z. Xie *et al.*, "Where to touch, how to contact: Hierarchical RL-MPC framework for geometry-aware long-horizon dexterous manipulation," *arXiv:2601.10930*, 2026.
[34] T. Pang *et al.*, "A convex quasistatic time-stepping scheme for rigid multibody systems with contact and friction," in *Proc. ICRA*, 2021, pp. 6614–6620.
[35] N. Hogan, "Impedance control: An approach to manipulation," in *Proc. ACC*, 1984, pp. 304–313.
[36] M. Anitescu *et al.*, "An iterative approach for cone complementarity problems for nonsmooth dynamics," *Comput. Optim. Appl.*, vol. 47, no. 2, pp. 207–235, 2010.
[37] M. Avriel, *Nonlinear programming: Analysis and methods*. Courier Corporation, 2003.
[38] J.-M. Lien *et al.*, "Approximate convex decomposition of polyhedra," in *Proc. SPM*, 2007, pp. 121–131.
[39] B. Acosta *et al.*, "Perceptive mixed-integer footstep control for under-actuated bipedal walking on rough terrain," *IEEE Trans. Robot.*, 2025.
[40] J. Nakanishi *et al.*, "Operational space control: A theoretical and empirical comparison," *Int. J. Robot. Res.*, vol. 27, pp. 737–757, 2008.
[41] G. Williams *et al.*, "Model predictive path integral control using covariance variable importance sampling," *arXiv:1509.01149*, 2015.
[42] S. Brahmbhatt *et al.*, "Contactdb: Analyzing and predicting grasp contact via thermal imaging," in *Proc. CVPR*, 2019, pp. 8709–8719.
[43] E. Todorov *et al.*, "MuJoCo: A physics engine for model-based control," in *Proc. IROS*, 2012, pp. 5026–5033.
[44] V. Makoviychuk *et al.*, "Isaac Gym: High performance GPU-based physics simulation for robot learning," arXiv:2108.10470, 2021.
[45] X. Wei *et al.*, "Approximate convex decomposition for 3D meshes with collision-aware concavity and tree search," *ACM Trans. Graph.*, vol. 41, no. 4, pp. 1–18, 2022.
[46] B. Wen *et al.*, "Foundationpose: Unified 6d pose estimation and tracking of novel objects," in *Proc. CVPR*, 2024, pp. 17 868–17 879.
[47] S. Yin *et al.*, "Visualmimic: Visual humanoid loco-manipulation via motion tracking and generation," *arXiv:2509.20322*, 2025.
[48] J. Kong *et al.*, "Learning soccer skills for humanoid robots: A progressive perception-action framework," *arXiv:2602.05310*, 2026.
[49] M. Lin *et al.*, "Bidexgrasp: Coordinated bimanual dexterous grasps across object geometries and sizes," *arXiv:2604.06589*, 2026.
[50] C. Borst *et al.*, "Grasp planning: How to choose a suitable task wrench space," in *Proc. ICRA*, vol. 1, 2004, pp. 319–325.
[51] M. Bhardwaj *et al.*, "Storm: An integrated framework for fast joint-space model-predictive control for reactive manipulation," in *Proc. CoRL*, 2022, pp. 750–759.
[52] W. Huang *et al.*, "Pointworld: Scaling 3D world models for in-the-wild robotic manipulation," *arXiv:2601.03782*, 2026.
[53] S. Xu *et al.*, "Interprior: Scaling generative control for physics-based human-object interactions," *arXiv:2602.06035*, 2026.